



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

Instytut Radiokomunikacji

Praca dyplomowa inżynierska

TRAVELAPP - APLIKACJA DO DOKUMENTOWANIA I PLANOWANIA PODRÓŻY

Adrian Oblizajek, 152035

Rafał Grudziak, 152043

Promotor

dr inż. Marcin Rodziewicz

POZNAŃ 2025

Streszczenie

Przedmiotem niniejszej pracy jest proces projektowania i implementacji aplikacji mobilnej TravelApp, umożliwiającej użytkownikom dokumentowanie i planowanie podróży. Projekt został zrealizowany w dwuosobowym zespole, w którym autorzy byli odpowiedzialni za różne aspekty techniczne: backend (serwer z API) zaimplementowany w PHP z wykorzystaniem frameworka Laravel oraz aplikację mobilną napisaną w Kotlinie z wykorzystaniem Jetpack Compose. W pracy opisano analizę rynku aplikacji podróżniczych oraz społecznościowych, proces implementacji i testowania, perspektywy dalszego rozwoju aplikacji oraz trudności napotkane w trakcie pracy.

Abstract

The subject of this paper is the process of designing and implementing TravelApp, a mobile application that allows users to document and plan their travels. The project was carried out in a two-person team, in which the authors were responsible for various technical aspects: the backend (server with API) implemented in PHP using the Laravel framework, and the mobile application written in Kotlin using Jetpack Compose. The paper describes an analysis of the travel and social networking application market, the implementation and testing process, the prospects for further development of the application and the difficulties encountered during the work.

Spis treści

Streszczenie	ii
Abstract	ii
1 Wstęp	2
1.1 Wprowadzenie	2
1.2 Zakres pracy	2
1.3 Struktura pracy	3
2 Technologie aplikacji	4
2.1 Technologie aplikacji mobilnej	4
2.2 Technologie serwera z API	5
3 Funkcjonalności aplikacji i analiza rynku	6
3.1 Analiza rynku aplikacji podróżniczych	6
3.1.1 Snapchat	6
3.1.2 TripAdvisor	8
3.2 Opis funkcjonalności	11
3.3 Diagramy przypadków użycia	12
3.4 Projekt architektury systemu	13
4 Implementacja	14
4.1 Implementacja aplikacji mobilnej	14
4.1.1 Struktura projektu aplikacji	14
4.1.2 Implementacja poszczególnych modułów aplikacji	19
4.1.3 Integracja aplikacji z API	29
4.1.4 Integracja aplikacji z mapami	33
4.2 Implementacja serwera z API	36
4.2.1 Struktura projektu	36
4.2.2 Opis punktów końcowych API	37
4.2.3 Opis implementacji punktu końcowego Timeline	41
4.2.4 Bezpieczeństwo i autoryzacja	44
4.3 Baza danych	45
4.3.1 Projekt bazy danych	45
4.3.2 Struktura bazy danych	47
5 Testowanie aplikacji i API	48
5.1 Testowanie aplikacji	48
5.2 Testowanie API	51

6	Wnioski i podsumowanie	53
6.1	Trudności napotkane w trakcie pracy	53
6.2	Perspektywy dalszego rozwoju aplikacji	55
6.3	Ocena osiągniętych celów	56
6.4	Wnioski z realizacji projektu	56
	Literatura	57
A	Diagramy przypadków użycia	59

Rafał Grudziak w ramach niniejszej pracy zaimplementował serwerowe API i bazę danych, opracował podrozdział 1.1 Wprowadzenie, 2.2 Technologie serwera z API, 3.1 Analiza rynku aplikacji podróźniczych, 3.4 Projekt architektury systemu, 4.2 Implementacja serwera z API, 4.3 Baza danych, 5.2 Testowanie API, 6.3 Ocena osiągniętych celów, 6.4 Wnioski z realizacji projektu.

Adrian Oblizajek w ramach niniejszej pracy zaimplementował aplikację mobilną, opracował podrozdziały 1.2 Zakres pracy, 1.3 Struktura pracy, 2.1 Technologie aplikacji mobilnej, 3.2 Opis funkcjonalności, 3.3 Diagramy przypadków użycia, 4.1 Implementacja aplikacji mobilnej, 5.1 Testowanie aplikacji, 6.1 Trudności napotkane w trakcie pracy, 6.2 Perspektywy dalszego rozwoju aplikacji.

Rozdział 1

Wstęp

1.1 Wprowadzenie

Celem pracy jest zaprojektowanie oraz stworzenie prostej i intuicyjnej aplikacji na urządzenia mobilne z systemem Android, służącej do dzielenia się swoimi podróżami, a także planowania przyszłych podróży. Aplikacja pozwala na logowanie i rejestrację użytkowników, wyszukiwanie użytkowników i dodawanie ich do znajomych, przeglądanie historii z podróży swoich oraz znajomych.

1.2 Zakres pracy

Aplikacja TravelApp została zaprojektowana z myślą o osobach, które chcą dokumentować swoje podróże i odkrywać nowe ciekawe miejsca na mapie świata. Aplikacja pozwala na czerpanie inspiracji dotyczących podróży od społeczności TravelApp. Użytkownicy mogą podzielić się swoimi doświadczeniami i wrażeniami z podróży z gronem znajomych.

Główne funkcjonalności aplikacji obejmują:

- Rejestrację i logowanie użytkownika;
- Edycję profilu użytkownika;
- Dodawanie i edytowanie wpisów z podróży;
- Reprezentację podróży na interaktywnej mapie od Mapbox;
- Przeglądanie profili i podróży innych użytkowników aplikacji;
- Wyszukiwanie dodanych podróży przez użytkowników aplikacji;
- Wyszukiwanie profili użytkowników;
- Interakcje społecznościowe - dodawanie użytkowników do znajomych, zarządzanie zaproszeniami do znajomych, dostęp do podróży znajomych.

Aplikacja TravelApp składa się z części frontend, która jest zaimplementowana w języku Kotlin i części backend napisanej w języku PHP z wykorzystaniem frameworka Laravel. Frontend odpowiedzialny jest m.in. za pobieranie danych z serwera i wyświetlanie ich w aplikacji mobilnej oraz za przysyłanie na serwer wprowadzonych w aplikacji danych przez użytkownika. Backend

odpowiada na zapytania aplikacji mobilnej o konkretne dane, np. dane użytkownika czy listę podróży. Serwer odpowiedzialny jest również za przetwarzanie przesłanych danych przez aplikację, oraz za zabezpieczenie danych, tak aby miały do nich dostęp tylko uprawnione osoby.

Zakres pracy związany z projektem aplikacji zawiera:

- Implementację aplikacji mobilnej TravelApp;
- Implementację i dokumentację serwerowego API;
- Implementację bazy danych;
- Integrację aplikacji mobilnej z API;
- Integrację aplikacji z mapami;
- Testowanie aplikacji i serwera z API.

Serwer z API i aplikację mobilną zaimplementowano przy użyciu takich technologii jak:

- **Laravel** - framework PHP, wykorzystany do stworzenia serwera z API i obsługi logiki aplikacji po stronie serwera;
- **MySQL** - baza danych użyta do przechowywania informacji m.in. o użytkownikach i podróżach;
- **Docker** - platforma służąca do konteneryzacji, wdrażania i uruchamiania aplikacji;
- **Kotlin** - język programowania wykorzystany do implementacji aplikacji mobilnej;
- **Jetpack Compose** - framework oparty na języku Kotlin wykorzystywany do stworzenia interfejsu użytkownika;
- **Retrofit** - biblioteka, która umożliwi komunikację z serwerem za pomocą HTTP;
- **Mapbox** - platforma udostępniająca interaktywne mapy oraz API, wykorzystana do integracji aplikacji z mapami.

Narzędzia, które zostały użyte podczas tworzenia aplikacji:

- **PhpStorm** - środowisko programistyczne PHP;
- **Android Studio** - środowisko programistyczne, umożliwia tworzenie aplikacji na platformę Android.

Więcej informacji odnośnie użytych narzędzi i technologii znajduje się w rozdziale 2 Technologie aplikacji.

1.3 Struktura pracy

Dalsza struktura pracy jest następująca. Rozdział drugi zawiera krótkie opisy użytych technologii w implementacji aplikacji i API. Trzeci rozdział opisuje analizę rynku aplikacji podróżniczych, opis funkcjonalności aplikacji, diagramy przypadków użycia i opis architektury systemu. W czwartym rozdziale została przedstawiona implementacja kolejnych części składowych aplikacji: aplikacji mobilnej, serwera oraz bazy danych. Rozdział piąty powstał, aby przedstawić metody testowania aplikacji mobilnej oraz serwera z API. Rozdział szósty zawiera trudności napotkane w trakcie implementacji aplikacji, pomysły i perspektywy na dalszy rozwój aplikacji, ocenę osiągniętych celów oraz wnioski z realizacji projektu.

Rozdział 2

Technologie aplikacji

2.1 Technologie aplikacji mobilnej

- **Android** - aplikacja została zaprojektowana na urządzenia mobilne z systemem Android, który składa się z jądra opartego na Linuksie. Firma rozwijająca system Android czyli Android Inc. została kupiona przez Google w 2005 roku, a sam system został uruchomiony na urządzeniach mobilnych w 2007 roku. W 2024 roku był najpopularniejszym systemem na urządzeniach mobilnych, stanowił ponad 70 procent rynku. Google ciągle wydaje nowe wersje systemu Android, w 2024 roku Android 14 miał ponad 30 procentowy udział w rynku [1]. Aktualnie najnowszą wersją systemu jest Android 15;
- **Android Studio** - zintegrowane środowisko programistyczne, wykorzystane do tworzenia, debugowania i testowania aplikacji mobilnej TravelApp. Edytor kodu pozwala na pracę z językami programowania takimi jak Kotlin, Java, C, C++. Android Studio posiada emulator, który pozwala testować aplikacje na różnych urządzeniach z systemem Android. Android Studio używa systemu kompilacji Gradle [2], który umożliwia dostosowywanie procesu kompilacji i tworzenie kilku plików APK dla aplikacji z różnymi funkcjami. Android Studio zawiera również narzędzia do profilowania wydajności, dzięki którym można monitorować wykorzystanie procesora i pamięci przez aplikację oraz wykrywać wycieki pamięci [3];
- **Kotlin** - statycznie typowany język programowania wykorzystany do implementacji aplikacji mobilnej TravelApp. Kotlin obsługuje programowanie obiektowe i programowanie funkcyjne. Kotlin wykorzystywany jest do programowania aplikacji na system Android. Kotlin jest interoperacyjny z językiem Java, oznacza to, że kod Javy można wywoływać z Kotlinu i odwrotnie [4];
- **Jetpack Compose** - interfejs użytkownika aplikacji TravelApp został zbudowany za pomocą frameworka Jetpack Compose, który pozwala w sposób deklaratywny tworzyć komponenty UI za pomocą zdefiniowanych w nim funkcji. Elementy UI tworzone są za pomocą funkcji z adnotacją `@Composable`. Funkcje `@Composable` przekształcane są za pomocą kompilatora Kotlin w elementy UI aplikacji [5];
- **Mapbox** - platforma udostępniająca mapy i usługi lokalizacyjne dla aplikacji internetowych, aplikacji mobilnych oraz dla sektora motoryzacyjnego. Mapbox obejmuje interfejsy API takie jak Static Images API, Geocoding API [6]. Integracja aplikacji z mapami pozwala na wyświetlenie podróży na interaktywnej mapie, wyszukiwanie adresów, pobieranie statycznych obrazów mapy;

- **Retrofit** - biblioteka umożliwiająca komunikację aplikacji z serwerowym API. Klient Retrofit odpowiedzialny jest za tworzenie zapytań HTTP i przesyłanie ich do serwera oraz za odbieranie odpowiedzi od serwera i przekazywanie ich do aplikacji.

2.2 Technologie serwera z API

W procesie implementacji wykorzystano szereg nowoczesnych technologii i narzędzi, które zapewniły aplikacji efektywne działanie, skalowalność i bezpieczeństwo. Najważniejsze z nich to:

- **Laravel** - framework PHP, który ułatwia tworzenie aplikacji internetowych za pomocą rozbudowanego zestawu funkcji, takich jak routing, middleware, wbudowana obsługa uwierzytelniania i autoryzacji czy wbudowana komunikacja z bazą danych. Laravel zapewnia czytelną strukturę kodu [7];
- **MySQL** - system zarządzania relacyjnymi bazami danych. Jest to jedno z najczęściej wykorzystywanych rozwiązań w połączeniu z Laravelem. [8];
- **Composer** - manager pakietów dla PHP, pozwalający na proste zarządzanie zależnościami. Użyty został do instalacji i aktualizacji bibliotek wykorzystywanych w backendzie, a także do zainstalowania samego Laravela [9];
- **Swagger/OpenAPI** - Swagger był wykorzystywany do testowania i dokumentowania punktów końcowych API. Umożliwia wygodne pisanie dokumentacji, która jest generowana w formie aplikacji internetowej, wyświetlającej przykładowe dane wysyłane do serwera z API, oraz przykładowe odpowiedzi serwera. Umożliwia łatwy dostęp do informacji o punktach końcowych oraz sprawne testowanie [10];
- **Docker** - platforma do konteneryzacji, umożliwiająca tworzenie i uruchamianie aplikacji w odizolowanych środowiskach kontenerowych. Docker pomagał uruchamiać serwer aplikacji wraz z bazą danych MySQL [11].

Rozdział 3

Funkcjonalności aplikacji i analiza rynku

3.1 Analiza rynku aplikacji podróżniczych

Rynek aplikacji podróżniczych jest dość zróżnicowany i obejmuje aplikacje różnego typu:

- Aplikacje ukierunkowane na rezerwacje noclegów - takie jak Booking, Airbnb, które umożliwiają użytkownikom wyszukiwanie i rezerwowanie miejsc noclegowych [12, 13];
- Aplikacje społecznościowe - takie jak Instagram, Snapchat, nie są one ukierunkowane dla podróżników, lecz na dzielenie się zdjęciami. Wiele osób wykorzystuje te aplikacje, aby podzielić się ze swoimi znajomymi zdjęciami z podróży [14, 15];
- Aplikacje do wyszukiwania miejsc - np. TripAdvisor, aplikacja umożliwiająca wyszukiwanie miejsc, przeglądanie oraz dodawanie recenzji hoteli, restauracji, atrakcji turystycznych oraz innych miejsc. Ta aplikacja oferuje również możliwość planowania podróży [16].

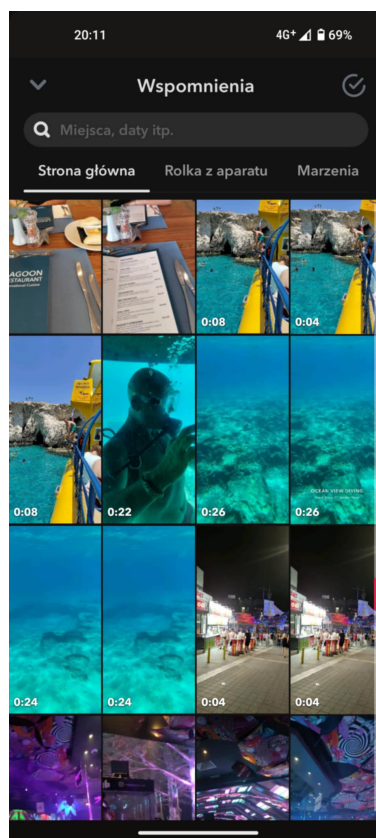
TravelApp to aplikacja podróżniczo-społecznościowa, która oferuje możliwość planowania podróży oraz umożliwia dzielenie się swoimi podróżami zawierającymi pamiątkowe zdjęcia ze swoimi znajomymi. Zatem analizie zostały poddane dwie aplikacje, pierwszą z nich jest aplikacja Snapchat, która jest aplikacją społecznościową, która umożliwia dzielenie się zdjęciami oraz filmikami ze znajomymi. Drugą aplikacją jest TripAdvisor, aplikacja służąca do planowania podróży, wyszukiwania ciekawych miejsc oraz sprawdzania opinii.

3.1.1 Snapchat

Snapchat to aplikacja społecznościowa, służąca do dzielenia się ze znajomymi zdjęciami oraz krótkimi filmikami, które po krótkim czasie znikają. Jest aplikacją ukierunkowaną na multimedia i szybką komunikację wizualną. Treści te można przysyłać bezpośrednio do swoich znajomych lub tworzyć "Stories", które są dostępne dla wszystkich znajomych przez 24 godziny od dodania i mogą być oglądane bez limitu. Snapchat oferuje również mapę, na której można zobaczyć aktualną lokalizację tych znajomych, którzy zezwolili na udostępnianie swojej lokalizacji. Kolejną ciekawą funkcjonalnością tej aplikacji jest zakładka "Memories", w której znajduje się archiwum "Snapów" (zdjęć, filmików) z datą utworzenia oraz lokalizacją. Dzięki temu użytkownicy mogą otworzyć mapę wraz ze "Snapami" wyświetlonymi jako pinezki w miejscach gdzie zostały wykonane [15].

Podobną funkcją aplikacji Snapchat do funkcjonalności zaimplementowanej w aplikacji TravelApp jest funkcja "Memories" czyli wspomnienia. Jedną z dwóch form tej funkcjonalności jest

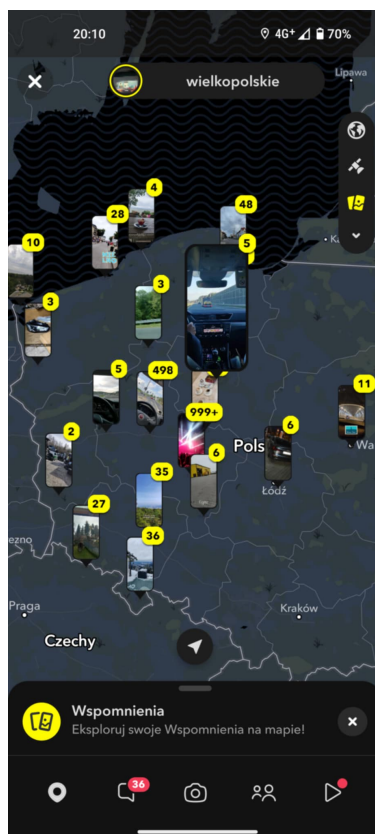
galeria wszystkich wykonanych i zapisanych zdjęć za pomocą aplikacji. Zdjęcia zawierają zapisaną lokalizację gdzie zostały wykonane. Funkcja pozwala przypomnieć najciekawsze chwile, co przedstawia rys. 3.1.



RYСУNEK 3.1: Wspomnienia w formie galerii w aplikacji Snapchat

W aplikacji TravelApp użytkownik ma dostęp do listy swoich podróży. Dopiero po wejściu w podróż, użytkownik może zobaczyć opis, odwiedzone miejsca oraz przypisane zdjęcia do podróży, czy dokładnie do danych miejsc.

Wspólną funkcjonalnością obu aplikacji jest mapa z pinezkami. W aplikacji TravelApp po kliknięciu w pinezkę użytkownik zobaczy informacje o podróży, w tym opis i zdjęcia. Aplikacja Snapchat posiada tę funkcjonalność bardziej rozwiniętą, pinezki wyświetlają zdjęcia z miejsc w formie miniaturowego pokazu slajdów oraz małą ikonkę z liczbą zdjęć z danego miejsca, co pokazuje rys. 3.2.



RYSUNEK 3.2: Wspomnienia w formie mapy w aplikacji Snapchat

Snapchat i TravelApp pozwalają na archiwizowanie wspomnień, zdjęć z danego miejsca, do których można wrócić w każdej chwili. Obie aplikacje umożliwiają wyszukiwanie innych profili i zapraszanie do znajomych w celu posiadania dostępu do udostępnianych materiałów przez inną osobę. W aplikacji Snapchat aby uzyskać dostęp do "Stories" znajomych lub wysyłać wzajemnie "Snapy". W TravelApp w celu uzyskania dostępu do podróży znajomych.

Różnice między Snapchat a TravelApp

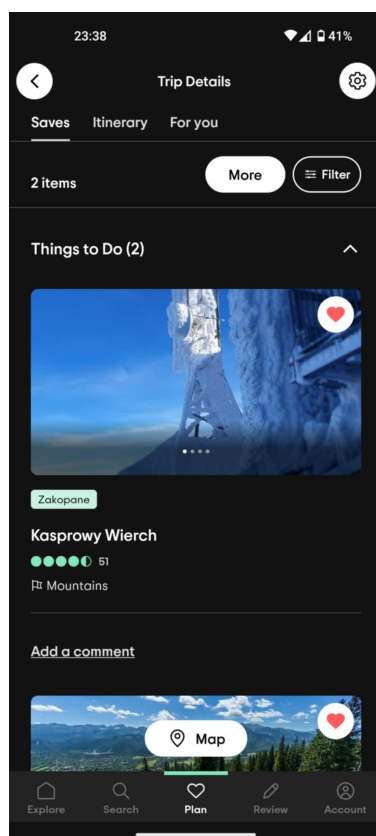
- **Przeznaczenie** - Snapchat jest aplikacją społecznościową do udostępniania ulotnych multimedów, a TravelApp jest aplikacją podróżniczo-społecznościową do planowania podróży oraz dzielenia się odwiedzionymi miejscami i zdjęciami z tych miejsc;
- **Trwałość treści** - w TravelApp, wszelkie treści dodane przez użytkownika są dostępne, zarówno dla właściciela jak i innych osób do momentu, aż nie zostaną usunięte przez właściciela. W aplikacji Snapchat, wysłane treści znikają po otwarciu, a dla właściciela są dostępne do momentu usunięcia o ile użytkownik zapisze multimedia w "Memories";
- **Planowanie i organizacja** - TravelApp oferuje możliwość zaplanowania podróży, dodania miejsc do odwiedzenia, oraz przypisania zdjęć do danej podróży. Snapchat nie posiada takich możliwości, jedynie na podstawie lokalizacji wykonania "Snapa", czyli zdjęcia lub nagrania filmiku, można znaleźć je później na mapie.

3.1.2 TripAdvisor

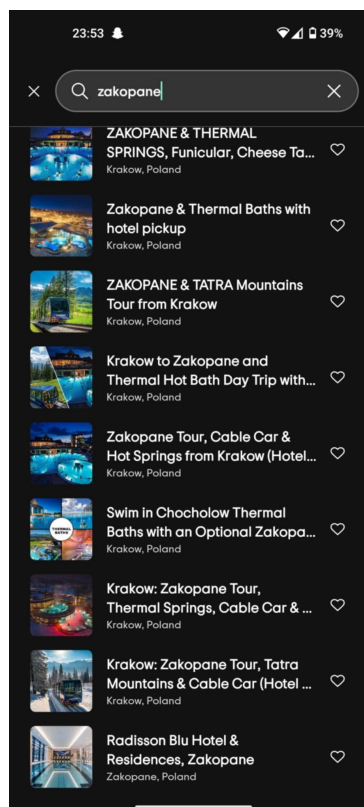
TripAdvisor to platforma ukierunkowana na zbieraniu opinii o miejscach. Często jest wykorzystywana podczas planowania podróży. Umożliwia odkrywanie ciekawych miejsc i dzielenie się

opiniami o odwiedzonych lokalizacjach przez użytkowników. Pozwala zobaczyć oceny i przeczytać opinie o atrakcjach, restauracjach, noclegach itp. Oferuje wiele możliwości eksplorowania świata za pomocą rekomendacji i ocen innych użytkowników. Dodatkowo aplikacja posiada funkcję planowania podróży, można zaplanować i dodać ciekawe miejsca do odwiedzenia podczas wyjazdu. Jest to świetne narzędzie do wyszukiwania ciekawych miejsc podczas tworzenia planu wycieczki. Aby jeszcze bardziej usprawnić użytkownikowi planowanie, aplikacja posiada funkcjonalność, która zaplanuje podróż dla użytkownika na podstawie wybranych preferencji, wykorzystując sztuczną inteligencję.

Aplikacja TripAdvisor podobnie jak TravelApp, pozwala stworzyć podróż oraz dodać listę miejsc do odwiedzenia. Rys. 3.3 przedstawia listę miejsc do odwiedzenia zaplanowanej podróży w aplikacji TripAdvisor. Miejsca wyszukuje się w wyszukiwarce, która przedstawiona jest na rys. 3.4. TripAdvisor posiada bazę miejsc, która jest przeszukiwana po wpisaniu odpowiedniej frazy. Natomiast TravelApp nie posiada własnej bazy miejsc, to użytkownik dodając miejsce do podróży, musi zaznaczyć lokalizację, którą chce odwiedzić i nazwać ją według własnych preferencji.

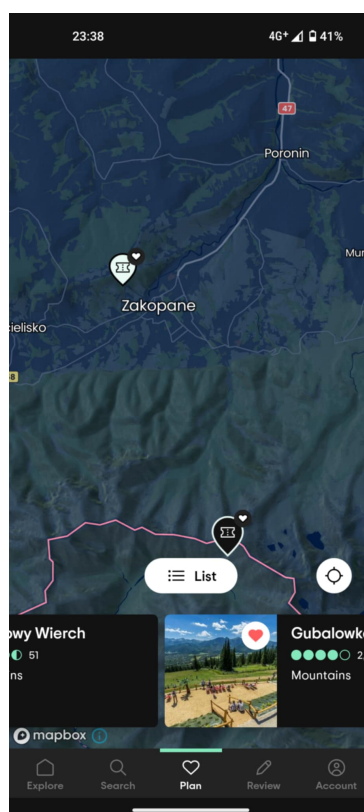


RYSUNEK 3.3: Zaplanowana podróż w aplikacji TripAdvisor



RYSUNEK 3.4: Wyszukiwanie miejsc w aplikacji TripAdvisor

TripAdvisor podobnie jak TravelApp oferuje widok mapy z pinezkami miejsc do odwiedzenia, co może pomóc zaplanować trasę i kolejność odwiedzania, przedstawione to zostało na rys. 3.5.



RYSUNEK 3.5: Widok mapy z wybranymi miejscami w aplikacji TripAdvisor

Różnice między TripAdvisor a TravelApp

- **Opinie i recenzje** - TripAdvisor jest aplikacją ukierunkowaną na wyszukiwanie miejsc i sprawdzanie opinii, pozwala na dodawanie ocen do miejsc, czego nie ma w aplikacji TravelApp, która jest aplikacją społecznościową;
- **Interakcje społecznościowe** - TravelApp oferuje wyszukiwanie profili oraz dodawanie do znajomych w celu przeglądania podróży, czego nie ma w aplikacji TripAdvisor;
- **Baza miejsc** - TripAdvisor posiada własną, obszerną bazę miejsc, co pozwala na łatwe wyszukiwanie i przeglądanie atrakcji turystycznych. W aplikacji TravelApp, to użytkownik samodzielnie dodaje miejsce zaznaczając jego lokalizację oraz wpisując nazwę i opis.

3.2 Opis funkcjonalności

Aplikacja mobilna TravelApp została zaprojektowana z myślą o osobach, które chciałyby nie tylko dokumentować i zapisywać podróże, ale również odkrywać nowe interesujące miejsca odwiedzone przez innych użytkowników oraz dzielić się swoimi doświadczeniami podróżniczymi.

Funkcjonalności zaimplementowane w aplikacji zostały podzielone na cztery moduły, które przedstawiają poszczególne funkcje, które aplikacja udostępnia użytkownikom.

A. Moduł logowania i rejestracji

Moduł pozwala użytkownikowi założyć nowe konto lub zalogować się do istniejącego konta w aplikacji. Logowanie do konta w aplikacji wymaga podania adresu e-mail i hasła. Użytkownik, który chce założyć nowe konto w aplikacji, musi podać adres e-mail, nazwę użytkownika i hasło. Użytkownik ma możliwość wylogowania się z konta. Moduł obejmuje nawigację między ekranami logowania i rejestracji.

Aplikacja poinformuje użytkownika, gdy poda niepoprawne dane podczas logowania lub rejestracji nowego konta w systemie.

B. Moduł profilu użytkownika

Aplikacja pozwala na wyświetlanie profilu zalogowanego użytkownika i przeglądanie profili innych użytkowników aplikacji. Profil użytkownika obejmuje takie informacje jak nazwa, opis, zdjęcie, preferencje podróżnicze, linki do mediów społecznościowych oraz statystyki profilu takie jak liczba odbytych podróży, liczba zaplanowanych podróży.

Użytkownik wyświetlając swój profil może wyświetlić listę znajomych lub zaproszenia do znajomych, które może akceptować lub usuwać. Użytkownik z poziomu listy znajomych może usunąć dodaną wcześniej osobę ze znajomych.

Z poziomu profilu można nawigować do galerii podróży użytkownika. Użytkownik może przejść do ekranu edycji swojego profilu. Edycja profilu obejmuje nazwę, zdjęcie, adres e-mail, opis, preferencje podróżnicze, linki do mediów społecznościowych oraz zmianę hasła.

Aplikacja po wykonaniu operacji sieciowej poinformuje użytkownika o jej wyniku np. dla edycji profilu, aplikacja przekazuje informację użytkownikowi o sukcesie lub wprowadzeniu niepoprawnych danych.

C. Moduł strony głównej

Strona główna aplikacji odpowiedzialna jest za pobieranie z serwera wpisów z podróży użytkowników aplikacji, udostępnia wyszukiwarkę znajomych i podróży z możliwością zastosowania dodatkowych filtrów dla wyników wyszukiwania podróży. Strona główna pobiera więcej informacji o podróży z serwera jeśli użytkownik przejdzie do podglądu danej podróży. Wyszukiwarka odpowiedzialna jest również za sortowanie wyników wyszukiwania podróży po dacie, która określa termin odbywania się podróży.

Przedstawione funkcjonalności strony głównej TravelApp pozwalają zalogowanemu użytkownikowi na przeglądanie wpisów z podróży, wyszukiwanie użytkowników po ich nazwie oraz wyszukiwanie podróży po sprecyzowanym zakresie dat. Użytkownik może przejść do profilu użytkownika, którego wcześniej wyszukał. Aplikacja uniemożliwia wyświetlanie wpisów z wycieczek użytkowników, którzy nie są znajomymi zalogowanego użytkownika, zarówno na stronie głównej, jak i w wyszukiwarce. System chroni w ten sposób dane użytkowników TravelApp.

Wyszukiwarka umożliwia przeprowadzenie akcji społecznościowych takich jak wysyłanie zaproszeń do znajomych użytkowników aplikacji oraz akceptacje lub usuwanie zaproszeń do znajomych. Strona główna pozwala na nawigację do profilu zalogowanego użytkownika oraz do formularza dodawania podróży.

D. Moduł podróży

Moduł podróży umożliwia użytkownikom aplikacji tworzenie i edycję wpisów z podróży. Formularz tworzenia i edycji podróży zawiera takie pola jak nazwę, opis, daty rozpoczęcia i zakończenia podróży, zdjęcia podróży, lokalizację oraz miejsca powiązane z daną podróżą. Miejsce składa się z nazwy, opisu, kategorii, lokalizacji i zdjęcia. Formularz podróży pozwala na dodanie, edycję lokalizacji podróży i dodanych miejsc przez ręczne zaznaczenie lokalizacji na mapie lub wyszukiwanie lokalizacji za pomocą dostępnej w aplikacji wyszukiwarki.

Galeria podróży pozwala użytkownikom na przeglądanie dodanych podróży, które zostały podzielone na trzy listy - zakończone, ulubione i zaplanowane podróże. Dostęp do galerii podróży jest ograniczony do znajomych danego użytkownika. Aplikacja została zintegrowana z mapami Mapbox, umożliwiając wyświetlanie podróży oraz miejsc w ramach podróży na interaktywnej mapie. Przedstawienie podróży na mapie pozwala na wizualizację i organizację odwiedzonych miejsc w ramach podróży użytkownika.

Moduł podróży udostępnia takie funkcje jak usuwanie wcześniej dodanych wpisów z wycieczek, dodawanie i usuwanie wpisu z listy ulubionych. Przedstawione funkcje aplikacji pozwalają użytkownikom TravelApp na zarządzanie treściami podróżniczymi w swojej galerii.

3.3 Diagramy przypadków użycia

Diagram przypadków użycia przedstawia, w jaki sposób użytkownicy wchodzi w interakcje z systemem. Diagram pozwala na opisanie, jak użytkownicy mogą korzystać z systemu oraz jakie działania może podejmować użytkownik [17].

Diagram przypadków użycia składa się z:

- Aktorzy - osoby, systemy zewnętrzne korzystające z funkcjonalności systemu.
- Przypadki użycia - interakcje użytkownika z aplikacją, funkcje systemu.
- Powiązania - linie łączące aktorów z danymi przypadkami użycia.

Diagramy przypadków użycia pozwoliły na przedstawienie funkcjonalności aplikacji. Umożliwiły one również uporządkowanie poszczególnych funkcji w odpowiednich modułach. Wykorzystanie diagramów pozwoliło na zdefiniowanie, w jaki sposób użytkownicy wchodzi w interakcje z aplikacją. W przypadku aplikacji TravelApp, aktorem będzie użytkownik korzystający z aplikacji. Przypadki użycia będą reprezentowane przez funkcjonalności aplikacji, np. logowanie do aplikacji, dodawanie nowego wpisu podróży. Szczegóły dotyczące diagramów przypadku użycia znajdują się w załączniku A.

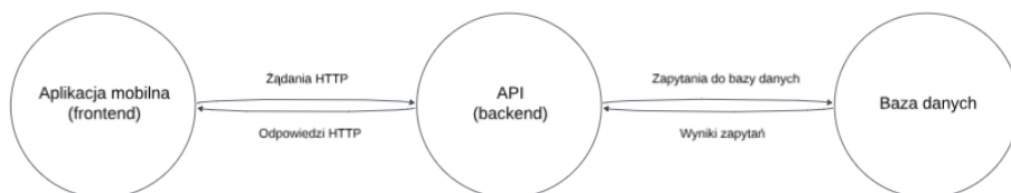
3.4 Projekt architektury systemu

Projekt architektury systemu aplikacji TravelApp opiera się na modelu klient-serwer, w którym aplikacja mobilna jest klientem komunikującym się z serwerowym API. Architektura ta pozwala na przetwarzanie danych na serwerze oraz przekazywanie klientowi tylko tych danych, do których może mieć dostęp, co zwiększa bezpieczeństwo systemu.

System składa się z trzech głównych komponentów:

- **Aplikacja mobilna** - aplikacja TravelApp na urządzenia z systemem Android, została napisana w języku Kotlin z wykorzystaniem Jetpack Compose. Do komunikacji z API, wykorzystywany jest Retrofit - biblioteka pozwalająca na wykonywanie żądań HTTP.
- **Serwer z API** - backend aplikacji został zaimplementowany w Laravelu, popularnym frameworku PHP, który dostarcza wiele funkcji ułatwiających implementację backendu w PHP.
- **Baza danych** - dane są przechowywane w bazie danych MySQL, która została zaprojektowana tak, aby przechowywać proste dane użytkowników, jak i strukturyzowane informacje o podróżach.

Diagram architektury systemu - rys. 3.6 przedstawia schemat architektury systemu.



RYSUNEK 3.6: Diagram architektury systemu

Opis przepływu danych

- Użytkownik wykonuje akcję w aplikacji mobilnej (np. wyświetlenie podróży);
- Retrofit wysyła do API żądanie (np. o dane podróży);
- API przetwarza żądanie i wykonuje odpowiednie zapytanie do bazy danych;
- API zwraca odpowiedź z odpowiednimi danymi;
- Retrofit przetwarza otrzymane dane;
- Dane zostają wyświetlone w interfejsie użytkownika.

Rozdział 4

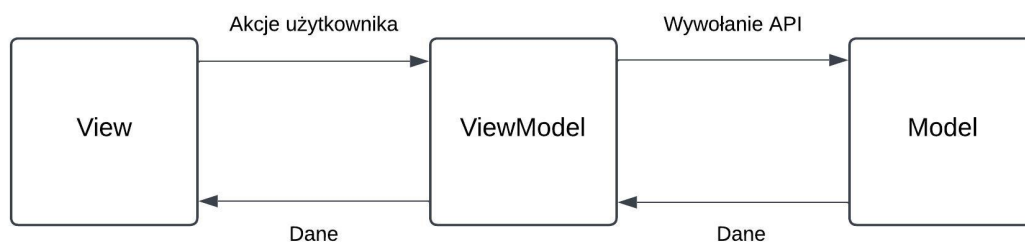
Implementacja

4.1 Implementacja aplikacji mobilnej

4.1.1 Struktura projektu aplikacji

W aplikacji zastosowano wzorec architektoniczny MVVM, czyli model, widok, model widoku. Wzorec MVVM opiera się na rozdzieleniu części graficznej czyli interfejsu użytkownika od logiki biznesowej odpowiedzialnej za komunikację z serwerowym API oraz za przetwarzanie danych otrzymanych z serwera (rysunek 4.1). Pośrednikiem we wzorcu MVVM między modelem a widokiem jest model widoku.

- **Widok** - graficzna reprezentacja interfejsu użytkownika. UI odpowiada za reprezentację danych na ekranie aplikacji, które zostały przekazane przez model widoku. Odpowiada za pobieranie danych od użytkownika, przesyła pobrane dane dalej do części modelu widoku;
- **Model widoku** - część pośrednicząca między widokiem a modelem. Odpowiedzialna za przetwarzanie danych odebranych od UI, zarządza stanem aplikacji na podstawie danych odebranych z serwera i od UI użytkownika. Wywołuje odpowiednie metody z repozytorium do komunikacji z API. Przetwarza klasy danych odebrane od Modelu i na ich podstawie ustawia odpowiedni stan UI, który zostanie wyświetlony użytkownikowi, przekazuje przetworzone dane do UI;
- **Model** - część biznesowa, odpowiada za reprezentację danych aplikacji za pomocą klas danych. Model odpowiedzialny jest również za interakcję z serwerowym API. Przetwarza odebrane dane od API i przekształca je w klasy danych, które następnie wysyłane są do modelu widoku. Model odbiera wywołania API z modelu widoku i komunikuje się z serwerowym API za pomocą odpowiednich metod.

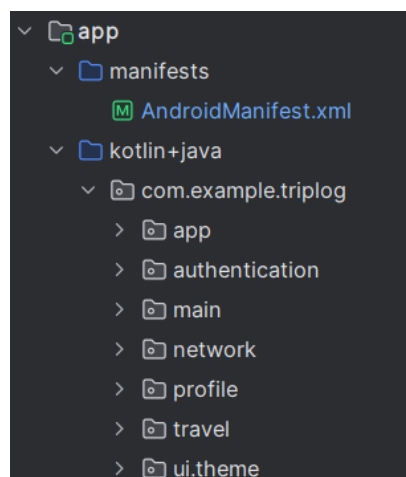


RYSUNEK 4.1: Wzorec architektoniczny MVVM

Struktura projektu aplikacji została zorganizowana w foldery przedstawiające poszczególne moduły aplikacji. Oprócz modułów aplikacji, które zostały omówione w poprzednim rozdziale, w strukturze znajdują się inne kluczowe foldery, takie jak *app*, *network*, *ui.theme*. Struktura została przedstawiona na rys. 4.2 i składa się z:

- **app** - folder zawierający klasę aplikacji, klasę *SessionManager* odpowiedzialną za zarządzanie sesją w aplikacji, klasę *AppNavigation* odpowiedzialną za nawigację w aplikacji oraz *ResponseHandler*, który zarządza odpowiedziami z serwera;
- **authentication** - folder zawiera dwa inne foldery *login* i *registration*, które przechowują klasy danych, modele widoku oraz UI związane z logowaniem i rejestracją użytkownika;
- **main** - folder zawiera klasy danych, model widoku oraz UI związane z stroną główną aplikacji;
- **network** - przechowuje klienta serwerowego API, repozytorium do komunikacji z API oraz klienta *MapboxClient*;
- **profile** - zawiera UI, klasy danych i modele widoku związane z profilem użytkownika i jego edycją;
- **travel** - folder składający się z folderów *travelGallery*, *travelForm* oraz *map*. Foldery przechowują UI, model widoku i klasy danych związane z formularzem podróży, galerią podróży użytkownika oraz z mapą, która jest istotnym elementem, ponieważ umożliwia wyszukiwanie lokalizacji i wizualizację podróży na mapie;
- **ui.theme** - przechowuje pliki odpowiedzialne za wygląd aplikacji. Pliki te odpowiadają za kolor, tło, typografię.

Podział struktury aplikacji na poszczególne moduły i kluczowe funkcje takie jak zarządzanie sesją, komunikację z API pozwala na organizację projektu i umożliwia prostą nawigację podczas rozbudowy aplikacji.



RYSUNEK 4.2: Struktura projektu aplikacji

Każdy z folderów danego modułu zawiera trzy foldery odnoszące się do MVVM. Folder *presentation* dotyczy UI czyli widoku, w nim znajdują się wszystkie elementy interfejsu użytkownika budujące dany moduł. Folder *ViewModel* zawiera modele widoku danego modułu. Folder *data* (część Model w MVVM) zawiera klasy danych danego modułu.

Przykładem zastosowania wzorca MVVM jest strona główna (rys. 4.3), która zbudowana jest z funkcji Composable *MainPageScreen* i modelu widoku *MainPageViewModel* (rys. 4.4 i 4.5). Ekran strony głównej wyświetla podróże znajomych, natomiast podczas operacji sieciowych takich jak pobieranie danych wyświetla wskaźnik ładowania. Model widoku strony głównej odpowiada za logikę ekranu i wywołania API, zawiera zmienne, które przechowują aktualny stan UI i dane takie jak lista wpisów podróży. Podczas inicjalizacji modelu widoku wywoływane jest API w celu pobrania podróży znajomych (rys. 4.6), klasa *Repository* odpowiada za przetworzenie odpowiedzi z Retrofit na klasę danych (rys. 4.7) - część reprezentująca model w MVVM. Klasa danych *TimelineResult* zwracana jest do modelu widoku, który wywołał funkcję z *Repository*.

```
composable(route = Screen.MainPageScreen.destination) {
    val viewModel: MainPageViewModel = viewModel(factory = MainPageViewModel.factory)
    MainPageScreen(
        viewModel = viewModel,
        navController = navController,
        sharedTravelViewModel = sharedTravelViewModel
    )
}
```

RYSUNEK 4.3: Ścieżka do ekranu strony głównej zdefiniowana w *NavHost*

```
@SuppressLint("StateFlowValueCalledInComposition")
@Composable
fun MainPageScreen(
    viewModel: MainPageViewModel,
    navController: NavController,
    sharedTravelViewModel: SharedTravelViewModel
) {
    LaunchedEffect(viewModel.loadingState, viewModel.mainPageState) {
        viewModel.handleLoadingState()
        viewModel.handleMainPageState()
    }

    BackHandler(enabled = true) {...}

    Scaffold(
        topBar = { MainPageTopBar(viewModel) },
        bottomBar = { MainPageBottomBar(navController, viewModel) },
    ) { innerpadding ->
        if (viewModel.isProgressIndicatorVisible) {
            FullScreenLoadingIndicator()
        } else {
            MainPageContent(innerpadding, viewModel, navController, sharedTravelViewModel)
        }
    }

    if (viewModel.isLogoutDialogVisible) {...}

    if (viewModel.isBackendResponseVisible) {...}
}
```

RYSUNEK 4.4: *MainPageScreen*

```
@SuppressLint("MutableCollectionMutableState")
class MainPageViewModel(
    private val repository: InterfaceRepository,
    val sessionManager: SessionManager,
    val responseHandler: ResponseHandler
) :
    ViewModel() {

    var mainPageState: MainPageState by mutableStateOf(MainPageState.Idle)
    var mainPageSection: MainPageSection by mutableStateOf(MainPageSection.Main)
    var loadingState: LoadingState by mutableStateOf(LoadingState.NotLoaded)
    var travelOverview by mutableStateOf(TravelData())
    var timelineTravels by mutableStateOf<List<TimelineTravel>>(emptyList())
    var searchedTravels by mutableStateOf<List<TimelineTravel>>(emptyList())
    var currentPageOfTimelineTravels by mutableIntStateOf(value: 1)
    var totalPagesOfTimelineTravels by mutableIntStateOf(value: 0)
    var selectedFilters by mutableStateOf(SearchFilters())
    var query by mutableStateOf(value: "")
    var searchedProfilesCurrentPage by mutableIntStateOf(value: 1)
    var searchedProfilesTotalPages by mutableIntStateOf(value: 0)
    var searchedTravelsCurrentPage by mutableIntStateOf(value: 1)
    var searchedTravelsTotalPages by mutableIntStateOf(value: 0)
    var searchedProfiles by mutableStateOf<List<SearchProfilesResult.Data>>(emptyList())

    val listState = LazyListState()
    private var authenticatedUserProfile: AuthenticatedUserProfileData by mutableStateOf(
        AuthenticatedUserProfileData()
    )

    var isProgressIndicatorVisible by mutableStateOf(value: false)
    var isBackendResponseVisible by mutableStateOf(value: false)
    var isLogoutDialogVisible by mutableStateOf(value: false)

    init {
        getAuthenticatedUserProfileData()
    }

    companion object {
        val factory: ViewModelProvider.Factory = viewModelFactory {...}
    }
}
```

RYSUNEK 4.5: Fragment klasy *MainPageViewModel*

```

fun loadTimelineTravels(
    page: Int = 1, dateFrom: String = "",
    dateTo: String = "", sortDirection: String = ""
) {
    loadingState = LoadingState.Loading
    val token = sessionManager.getToken()
    viewModelScope.launch {
        try {
            val result = repository.getTimeline(token, page, dateFrom, dateTo, sortDirection)
            if (result != null) {
                if (result.resultCode == 200 && result.data != null) {
                    timelineTravels = result.data
                    currentPageOfTimelineTravels = page
                    totalPagesOfTimelineTravels = result.meta?.lastPage ?: 1
                } else if (result.resultCode == 401 && result.message != null) {
                    val backendResponse = BackendResponse(message = result.message)
                    processUnauthenticatedState(backendResponse)
                } else {
                    val backendResponse = BackendResponse(message = result.message)
                    processErrorState(backendResponse)
                }
            }
        } catch (e: Exception) {...}
        loadingState = LoadingState.Loaded
    }
}

```

RYSUNEK 4.6: Funkcja *loadTimelineTravels* z modelu widoku *MainPageViewModel*

```

override suspend fun getTimeline(
    token: String?, page: Int?,
    dateFrom: String?, dateTo: String?,
    sortDirection: String?
): TimelineResult? {
    val response: Response<TimelineResult> =
        travelAppApiService.getTimeline(token: "Bearer $token", page, dateFrom, dateTo, sortDirection)

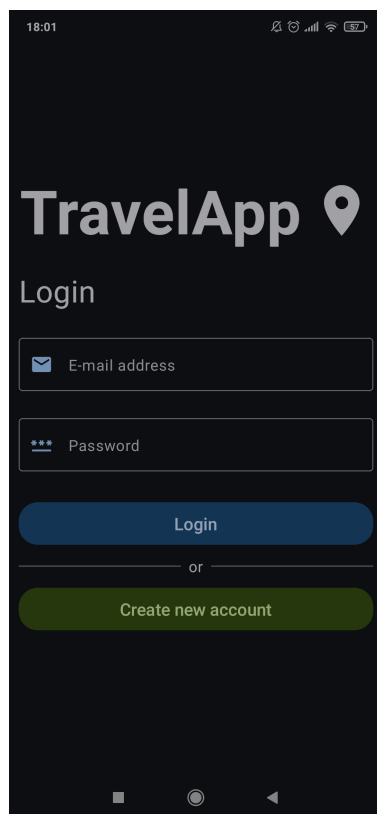
    val timelineResult: TimelineResult?
    if (response.isSuccessful) {
        timelineResult = response.body()
        timelineResult?.resultCode = response.code()
        return timelineResult
    } else {
        val errorBody = response.errorBody()?.string()
        timelineResult =
            Moshi.Builder().build().adapter(TimelineResult::class.java)
                .fromJson(errorBody!!)
        timelineResult?.resultCode = response.code()
        return timelineResult
    }
}

```

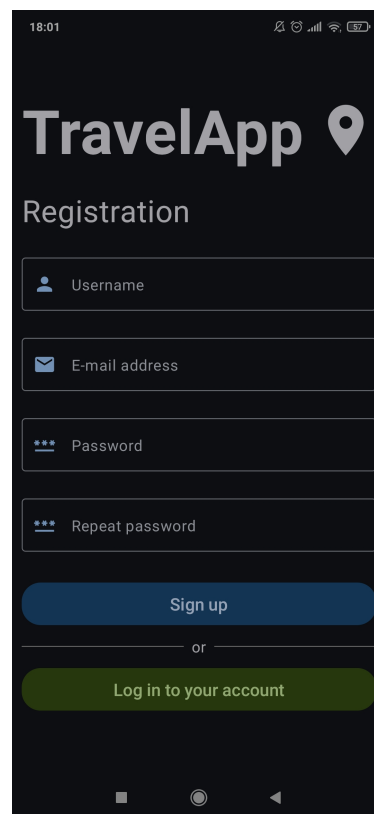
RYSUNEK 4.7: Przetworzenie pobranych danych na klasę danych *TimelineResult* w *Repository*

4.1.2 Implementacja poszczególnych modułów aplikacji

Pierwszym modulem, który został zaimplementowany, jest moduł logowania i rejestracji. Moduł odpowiada za tworzenie nowego konta i logowanie do istniejącego konta w aplikacji. Składa się z dwóch ekranów: ekran logowania i ekran rejestracji (rys. 4.8a i rys. 4.8b). Każdy ekran posiada model widoku odpowiedzialny za zarządzanie stanem UI i wywoływanie API z repozytorium *Repository*. Po zalogowaniu się lub zarejestrowaniu nowego konta aplikacja przenosi użytkownika do strony głównej aplikacji. Z ekranu logowania można nawigować do ekranu rejestracji i odwrotnie. W przypadku podania błędnych danych przy logowaniu lub rejestracji zostanie wyświetlony odpowiedni komunikat.



4.8A. Ekran logowania



4.8B. Ekran rejestracji

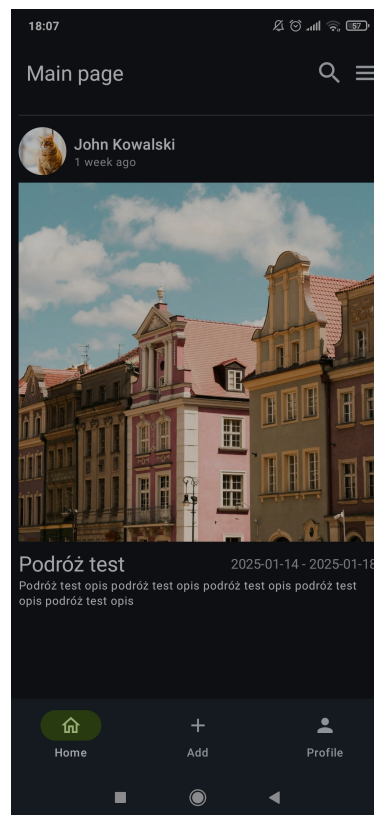
Kolejny moduł to strona główna aplikacji. Strona główna, jeśli użytkownik jest zalogowany w aplikacji, pobiera i wyświetla z serwera podróże znajomych użytkowników. Jeśli użytkownik nie jest zalogowany, to zostaje przeniesiony do ekranu logowania. Strona główna udostępnia wyszukiwarkę podróży i użytkowników, pozwala na nawigację do formularza podróży i do profilu zalogowanego użytkownika za pomocą dolnego paska nawigacyjnego, który zawiera odpowiednio przyciski *Home* - strona główna, *Add* - formularz, *Profile* - profil zalogowanego użytkownika. Dolny pasek nawigacyjny umożliwia użytkownikowi przemieszczanie się między głównymi modułami aplikacji. Dodatkowo z górnego paska nawigacyjnego zalogowany użytkownik może się wylogować z aplikacji.

Głównymi składowymi modułu strony głównej jest funkcja Composable *MainPageScreen* i *MainPageViewModel*, które zostały przedstawione w podrozdziale 4.1.1. *MainPageScreen* odpowiada za wyświetlanie wpisów podróży, wyszukiwarki, podglądu podróży, natomiast model widoku odpowiada za logikę strony głównej, ustawia stan strony głównej i wysyła zapytania do API np. w celu pobrania listy podróży, wyszukania użytkowników o podanej nazwie czy wylogowania użytkownika z aplikacji.

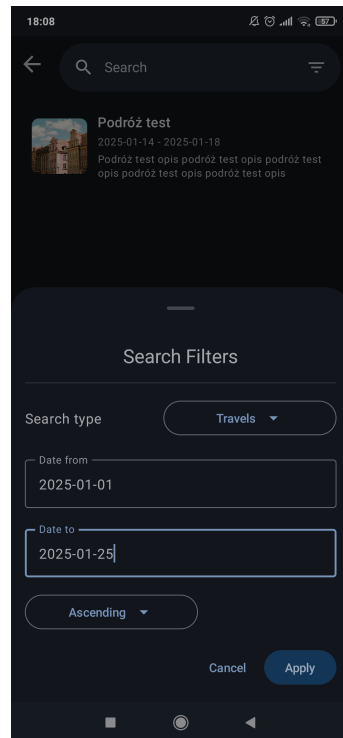
Ekran strony głównej *MainPageScreen* składa się trzech części:

- **Strona główna** - wyświetla wpisy podróży znajomych;
- **Podgląd podróży** - wyświetla szczegóły podróży i miejsc. Z podglądu podróży można nawigować do widoku podróży na mapie, który jest osobnym ekranem;
- **Wyszukiwarka** - umożliwia wyszukiwanie użytkowników i podróży użytkowników. Z wyszukiwarki użytkownik może wykonywać operacje społecznościowe takie jak zarządzanie zaproszeniami, dodawanie do znajomych. Użytkownik może również nawigować do profilu wybranego użytkownika.

Poszczególne elementy UI ekranu strony głównej zostały przedstawione na rys. 4.9, 4.10a oraz 4.10b.



RYSUNEK 4.9: Ekran strony głównej



4.10A. Wyniki wyszukiwania podróży z określonego zakresu dat



4.10B. Wyniki wyszukiwania użytkownika

Wyniki wyszukiwania użytkowników reprezentowane są w formie listy przy użyciu *LazyColumn* - rys. 4.11. Każdy element przewijalnej listy reprezentowany jest przez element UI *UserItem*, który wywołuje odpowiednie funkcje z modelu widoku w zależności od dostępnych opcji - akceptacja zaproszenia, odrzucenie zaproszenia, dodanie do znajomych, przejście do profilu użytkownika. Wyniki wyszukiwania podróży zostały zaimplementowane w podobny sposób z wykorzystaniem *LazyColumn* - rys. 4.12. UI reaguje na kliknięcie w wpis z listy i wywołuje z modelu widoku metodę odpowiedzialną za wczytanie szczegółów podróży przy jednoczesnej zmianie widoku ekranu na podgląd podróży.

```
when (viewModel.selectedFilters.searchType) {
    "Users" -> {
        LazyColumn {
            items(viewModel.searchedProfiles ?: emptyList()) { profile ->
                UserItem(
                    profile,
                    onClickReject = {
                        profile.receivedRequestId?.let { viewModel.rejectFriendRequest(it) }
                    },
                    onClickAccept = {
                        profile.receivedRequestId?.let { viewModel.acceptFriendRequest(it) }
                    },
                    onClickSendInvitation = {
                        profile.id?.let { viewModel.sendFriendRequest(it) }
                    },
                    onClick = {
                        navController.navigate(
                            route = "${Screen.ProfileScreen.destination}/${profile.id}"
                        )
                    }
                )
            }
        }
    }
}
```

RYSUNEK 4.11: Przewijalna lista reprezentująca wyniki wyszukiwania użytkowników

```

"Travels" -> {
  LazyColumn(
    modifier = Modifier
      .fillMaxSize()
      .padding(bottom = 64.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp)
  ) {
    items(viewModel.searchedTravels) { travel ->
      SearchedTravelItem(travel, onClick = {
        viewModel.travelOverview.id = travel.travel?.id
        viewModel.showTravel()
      })
    }
  }
}

```

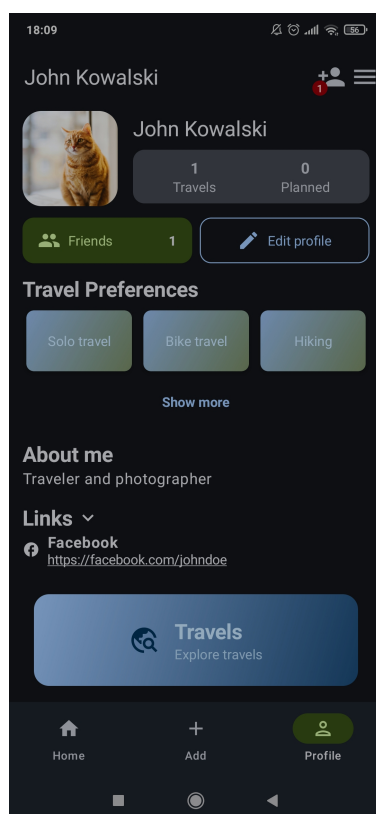
RYSUNEK 4.12: *LazyColumn* wyświetla wyniki wyszukiwania podróży

Moduł profilu zbudowany jest z dwóch funkcji Composable *ProfileScreen* i *EditProfileScreen*. Dodatkowo ekrany posiadają własne modele widoków czyli *ProfileViewModel* i *EditProfileViewModel*. *ProfileViewModel* odpowiedzialny jest za logikę ekranu profilu. Odpowiada za sprawdzenie, czy wczytywany profil jest profilem zalogowanego użytkownika. Jest to istotne, w kontekście wyświetlania i dostępności elementów UI odpowiedzialnych za edycję profilu, listę znajomych i listę zaproszeń, które powinny być dostępne tylko podczas przeglądania swojego profilu w aplikacji.

Ekran profilu *ProfileScreen* składa się z trzech części:

- **Profil użytkownika** - przedstawia profil użytkownika w aplikacji - rys. 4.13. Zalogowany użytkownik, wyświetlając swój profil ma możliwość nawigacji do edycji profilu, wyświetlania listy znajomych i listy zaproszeń do znajomych. Użytkownik również może przejść do galerii podróży, która jest osobnym ekranem;
- **Lista znajomych** - wyświetla znajomych użytkownika (rys. 4.14a). Lista udostępnia możliwość nawigacji do profilu znajomego. Użytkownik może usunąć znajomego z listy znajomych;
- **Lista zaproszeń** - wyświetla zaproszenia do znajomych - rys. 4.14b . List udostępnia zarządzanie zaproszeniami (akceptacja, usunięcie zaproszenia) i nawigację do profilu użytkownika, który wysłał zaproszenie. Po zaakceptowaniu zaproszenia, użytkownik zostanie dodany do listy znajomych, a podróże obu użytkowników będą dla nich widoczne.

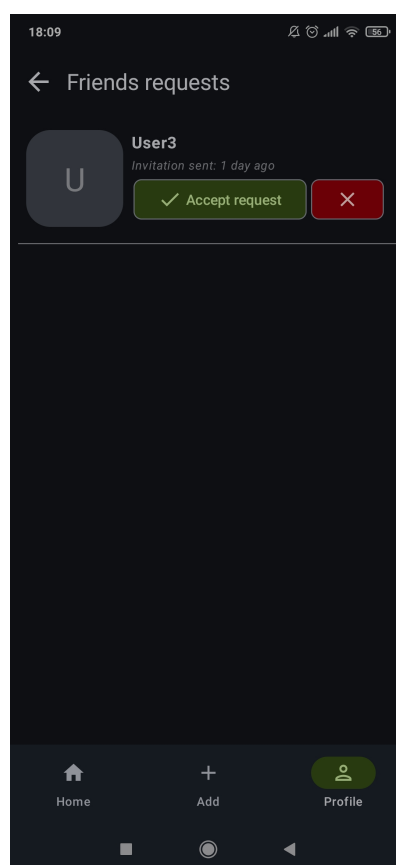
Zalogowany użytkownik wyświetlając profil znajomego w aplikacji może zobaczyć galerię podróży. Zalogowany użytkownik wchodząc na profil użytkownika, który nie jest jego znajomym, może tylko wyświetlić profil danego użytkownika bez możliwości przeglądania galerii podróży.



RYSUNEK 4.13: Profil zalogowanego użytkownika



4.14A. Lista znajomych



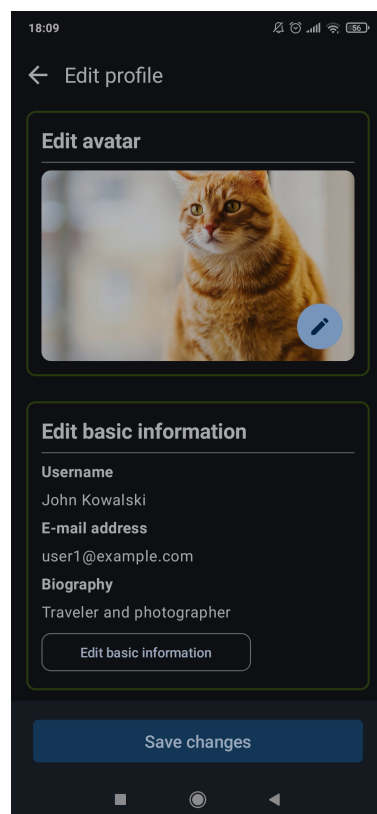
4.14B. Lista zaproszeń do znajomych

Pobrana lista znajomych użytkownika wykorzystywana jest przez *LazyColumn* (rys. 4.15), która wczytuje elementy listy i wyświetla wszystkie elementy w postaci przewijalnej listy. Każdy element listy reprezentowany jest za pomocą napisanej funkcji Composable *FriendItem*, która jest elementem UI reprezentującym pojedynczy wpis w liście.

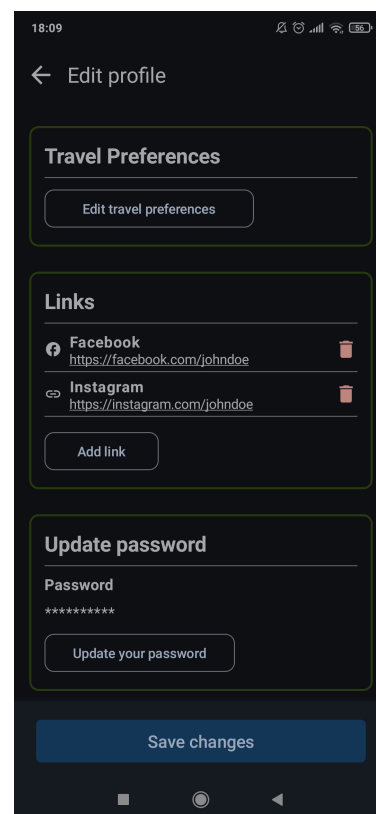
```
LazyColumn(
    modifier = Modifier
        .fillMaxSize()
        .padding(8.dp),
    contentPadding = innerpadding,
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Top
) {
    item {
        viewModel.friendsList.forEach { item ->
            FriendItem(
                item,
                onClickDelete = {
                    viewModel.isDeleteFriendDialogVisible = true
                    item?.id?.let { friendToDelete.value = it }
                },
                onClick = { navController.navigate( route: "${Screen.ProfileScreen.destination}/${item?.id}" ) })
            Spacer(modifier = Modifier.padding(vertical = 6.dp))
        }
    }
}
```

RYSUNEK 4.15: Implementacja listy znajomych

Edycja profilu przedstawia dostępne dane o profilu, które można edytować (rys. 4.16a i 4.16b). Umożliwia edycję zdjęcia, podstawowych informacji o użytkowniku (nazwa, opis, adres e-mail), dodanie preferencji podróżniczych z listy, dodanie linków do mediów społecznościowych użytkownika, aktualizację hasła.



4.16A. Ekran edycji profilu

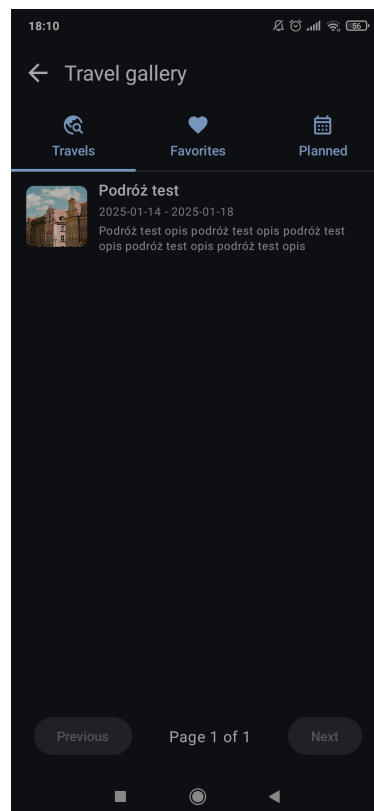


4.16B. Ekran edycji profilu

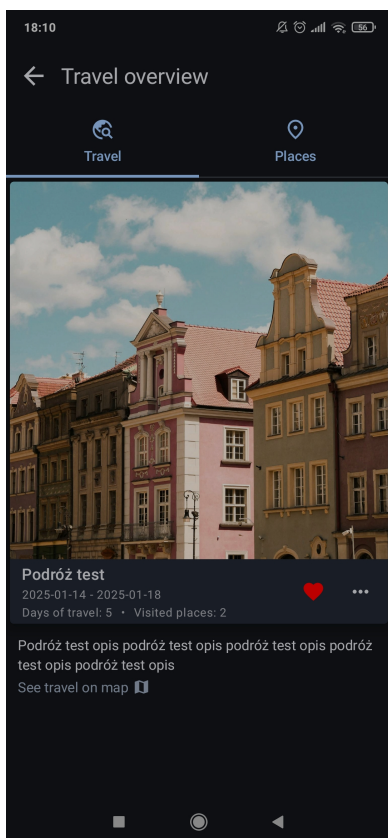
Kolejnym modulem jest moduł podróży, który obejmuje zarówno galerię podróży użytkownika, jak i formularz podróży wykorzystywany przy tworzeniu i edycji podróży.

Ekran galerii podróży składa się z kilku składowych:

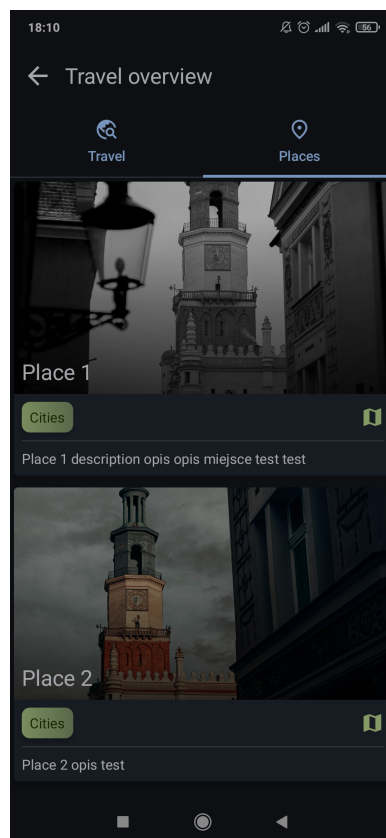
- **Galeria podróży** - składa się z trzech list - zakończone, ulubione i zaplanowane podróże. Mechanizm pobierania listy podróży został omówiony w podrozdziale 4.1.3 Integracja aplikacji z API. Rys. 4.17 przedstawia przykładową listę podróży. Użytkownik może przejść do podglądu podróży klikając w daną podróż;
- **Podgląd podróży** - przedstawia szczegóły podróży oraz listę miejsc w ramach podróży (rys. 4.18a i 4.18b). Podgląd własnej podróży umożliwia takie opcje jak usunięcie, edycję podróży, dodanie podróży do listy ulubionych. Z podglądu podróży użytkownik może przejść do widoku mapy, na której cała podróż zostanie umieszczona w formie znaczników;
- **Mapa podróży** - osobny ekran przedstawiający podróż w formie znaczników na interaktywnej mapie - rys. 4.19a. Klikając w znacznik użytkownik może wyświetlić szczegóły danej lokalizacji (rys. 4.19b).



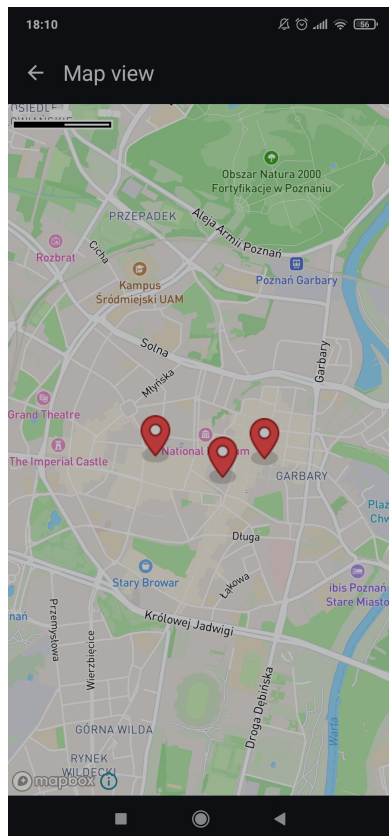
RYSUNEK 4.17: Galeria podróży



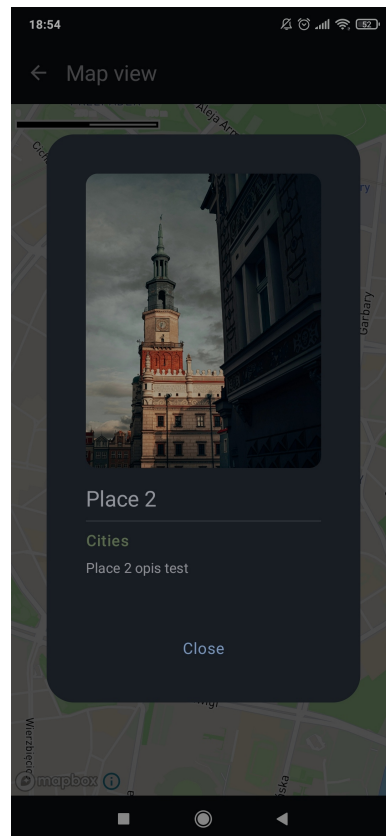
4.18A. Podgląd podróży



4.18B. Lista miejsc podróży



4.19A. Podróż na mapie



4.19B. Szczegóły znacznika na mapie

Fragment kodu odpowiedzialny za wczytanie mapy do *AndroidView* i wyświetlenie wszystkich punktów za pomocą *pointAnnotationManager* przedstawia rys. 4.20. Do współrzędnych każdego punktu dodawany jest marker w celu prezentacji wszystkich lokalizacji w ramach podróży. Kliknięcie punktu wczytuje odpowiednie dane do wyświetlenia użytkownikowi w formie dialogu. Dodatkowo, wcześniej ustawiany jest wybrany styl, w którym jest wyświetlana mapa w aplikacji.

```
AndroidView(  
    factory = { context ->  
        MapView(context).also { mapView ->  
            pointAnnotationManager = mapView.annotations.createPointAnnotationManager()  
            mapView.getMapboxMap().loadStyleUri(Style.MAPBOX_STREETS) { style ->  
                pointAnnotationManager.let {  
                    points.forEach { point ->  
                        val pointAnnotationOptions = PointAnnotationOptions()  
                            .withPoint(  
                                Point.fromLngLat(  
                                    point.point!!.longitude(),  
                                    point.point!!.latitude()  
                                )  
                            )  
                            .withIconImage(marker)  
                        pointAnnotationManager.create(pointAnnotationOptions)  
                    }  
                }  
            }  
        }  
    }
```

RYSUNEK 4.20: Fragment implementacji *MapScreen* odpowiedzialny za wczytanie mapy do *AndroidView*

Formularz podróży pozwala użytkownikowi dodać nową podróż do galerii lub edytować istniejącą podróż z galerii podróży. Do formularza użytkownik może przejść tak jak do poprzednich głównych ekranów za pomocą dolnego paska nawigacyjnego.

Ekran formularza podróży składa się z dwóch części:

- **Formularz podróży** - przedstawiony na rys. 4.21a i 4.21b, umożliwia uzupełnienie informacji o podróży takich jak nazwa, opis, data, lokalizacja, zdjęcia oraz lista miejsc. Miejsca do podróży można dodawać za pomocą ikony "plusa" w formularzu podróży, który jest widoczny na rys. 4.21a;
- **Formularz miejsca** - umożliwia uzupełnienie informacji o miejscu, które składa się z nazwy, opisu, kategorii, lokalizacji i zdjęcia. Rys. 4.22a, 4.22b przedstawiają formularz wykorzystywany przy tworzeniu, edycji miejsca podróży.

18:33

← Create new travel

Travel Places

Complete the information about your travel

Basic information

Travel name
Poznań

Travel description
Opis podróży test

Date of travel
07-01-2025 — 09-01-2025

Edit basic information

Travel places +

1 places added

Add new travel

4.21A. Formularz podróży

18:33

← Create new travel

Travel Places

Travel photo

Add new travel

4.21B. Formularz podróży

18:32

← New place

Basic information

Place name
Place

Place description
Opis miejsca test

Category of place
Cities

Edit basic information

Place photo

Add new place

4.22A. Formularz miejsca

18:32

← New place

Edit basic information

Place photo

Add new place

4.22B. Formularz miejsca

Z obu formularzy można nawigować do ekranu wyszukiwania lokalizacji, który zawiera takie funkcje jak wyszukiwanie lokalizacji na podstawie wpisanego adresu, ręczne ustawienie znacznika na mapie, pobranie adresu na podstawie współrzędnych. Wyszukiwanie lokalizacji posiada własny ekran *SearchMapScreen* i model widoku - *SearchMapViewModel*. Mechanizm wyszukiwania lokalizacji miejsca został przedstawiony w podrozdziale 4.1.4. Mapa statyczna widoczna na rys. 4.21b, 4.22a i 4.22b jest wczytanym przez aplikację obrazem mapy. Aby obraz został wyświetlony, aplikacja komunikuje się z API Mapboxa w celu pobrania adresu URL obrazu statycznej mapy dla danych współrzędnych. Implementacja statycznej mapy została przedstawiona na rys. 4.23 - *StaticMapView*. URL mapy pobierane jest dla danych współrzędnych (podróży lub miejsca) przez metodę *getStaticMapUrl* - rys. 4.24.

```
@Composable
fun StaticMapView(mapUrl: String?) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .aspectRatio(ratio: 16 / 9f),
        contentAlignment = Alignment.Center
    ) {
        mapUrl?.let {
            AsyncImage(
                model = it,
                contentDescription = null,
                modifier = Modifier.fillMaxSize(),
                contentScale = ContentScale.Crop
            )
        } ?: Text(stringResource(R.string.failedToLoadMap), color = Color.Red)
    }
}
```

RYSUNEK 4.23: Obraz mapy wczytany z URL pobranego z API Mapboxa

```
suspend fun getStaticMapUrl(
    longitude: Double?, latitude: Double?,
    marker: String, accessToken: String
): String {
    if (longitude != null && latitude != null) {
        val response = mapboxService.getStaticMap(
            lon = longitude, lat = latitude,
            marker = marker, accessToken = accessToken
        )
        if (response.isSuccessful) {
            return response.raw().request.url.toString()
        } else {
            throw Exception("Failed to fetch static map: ${response.errorBody()?.string()}")
        }
    }
    return ""
}
```

RYSUNEK 4.24: Metoda *getStaticMapUrl* pobiera URL obrazu mapy na podstawie przekazanych współrzędnych

4.1.3 Integracja aplikacji z API

Aplikacja TravelApp komunikuje się z serwerowym API za pomocą biblioteki Retrofit. Komunikacja aplikacji z API jest kluczowym elementem w działaniu całego systemu, odpowiedzialna jest za wymianę danych między aplikacją a serwerem, pozwala na pobieranie danych z serwera i wysyłanie danych w celu ich przetworzenia przez backend. Implementacja komunikacji umożliwia

przeprowadzanie takich operacji sieciowych jak pobranie informacji o profilu użytkownika, wysłanie danych podróży i ich zapis w bazie danych, wprowadzanie zmian w profilu, usuwanie podróży.

Pierwszym krokiem integracji aplikacji z API jest dodanie uprawnienia umożliwiającego aplikacji na nawiązywanie połączeń sieciowych i wykonywania zapytań HTTP. Uprawnienie zostało dodane do *AndroidManifest.xml* i zostało przedstawione na rys. 4.25.

```
<uses-permission android:name="android.permission.INTERNET" />
```

RYSUNEK 4.25: Uprawnienie umożliwiające dostęp do internetu

Następny krok obejmuje dodanie kilku zależności do projektu w Gradle Scripts (dokładnie *build.gradle.kts*), które obejmują Retrofit, Moshi i Coroutines. Niezbędne zależności zostały przedstawione na rys. 4.26.

```
implementation(libs.retrofit2.retrofit)
implementation(libs.retrofit2.converter.moshi)
implementation(libs.moshi.kotlin)
implementation(libs.moshi)
kapt(libs.moshi.kotlin.codegen)
implementation(libs.kotlinx.coroutines.test)
```

RYSUNEK 4.26: Zależności dodane do projektu do komunikacji z API

Retrofit odpowiada za komunikację z API, realizuje żądania HTTP do API i odbiera odpowiedzi z serwera. Moshi to konwerter danych JSON na obiekty Kotlin. Coroutines umożliwiają wykonywanie kodu w sposób asynchroniczny, jest to mechanizm wykorzystywany podczas wykonywania operacji sieciowych - pobierania, wysyłania danych między aplikacją a API.

W celu zintegrowania aplikacji i API zaimplementowano interfejs *TravelAppApiService*, który zawiera metody komunikacji z serwerem dla różnych funkcji systemu. Metody te wykorzystywane są do komunikacji z konkretnymi punktami końcowymi API. Każda metoda składa się z jednej z adnotacji *@GET*, *@POST*, *@PUT* lub *@DELETE*.

- *@GET* - pobieranie danych,
- *@POST* - wysyłanie danych,
- *@PUT* - aktualizacja istniejących danych,
- *@DELETE* - usuwanie danych.

Jedną z metod zaimplementowanych w *TravelAppApiService* jest metoda *getUserProfileResult* przedstawiona na rys. 4.27. Metoda *getUserProfileResult* odpowiedzialna jest za pobranie danych o profilu użytkownika. Oprócz adnotacji *@GET* oznaczającej pobranie danych, posiada również adnotację *@Header* i *@Path*. *@GET* zawiera punkt końcowy, czyli adres URL, który określa zasób, funkcję serwera, do których aplikacja wysyła zapytanie HTTP. *@Header* to nagłówek HTTP, który przesyła dodatkowe informacje między klientem a serwerem, w przypadku opisywanej metody z rys. 4.27 nagłówek przenosi token autoryzacyjny danego użytkownika, który zapewnia autoryzację użytkownika w systemie. Oprócz *@Header*, metoda pobierająca dane użytkownika zawiera *@Headers*, która określa format przesyłanych i odbieranych danych, w tym przypadku jest to format JSON. Adnotacja *@Path* umożliwia przekazanie danych przez ścieżkę URL, w opisywanym

przypadku jest to parametr "user", który określa konkretnego użytkownika systemu. W ścieżce URL "user" zostanie zastąpiony zmienną "id".

```
@Headers("Content-Type: application/json", "Accept: application/json")
@GET("/api/profiles/{user}")
suspend fun getUserProfileResult(
    @Header("Authorization") token: String,
    @Path("user") id: Int
): Response<UserProfileResult>
```

RYSUNEK 4.27: Metoda *getUserProfileResult*

Następnym elementem integracji aplikacji z API jest klasa klienta - *TravelAppRetrofitClient*. *TravelAppRetrofitClient* został przedstawiony na rys. 4.28 i zawiera adres URL serwera, konfiguruje klienta Retrofit oraz przekazuje instancje *TravelAppApiService* zawierającą wszystkie metody dostępne w API do repozytorium *Repository*.

```
interface RepositoryContainer {
    val repository: InterfaceRepository
}

class TravelAppRetrofitClient : RepositoryContainer {
    private val url = "http://adresApi/"

    private val moshi: Moshi = Moshi.Builder()
        .add(KotlinJsonAdapterFactory())
        .build()

    private val retrofit: Retrofit = Retrofit.Builder()
        .baseUrl(url)
        .addConverterFactory(MoshiConverterFactory.create(moshi).asLenient())
        .build()

    private val retrofitService: TravelAppApiService by lazy {
        retrofit.create(TravelAppApiService::class.java)
    }

    override val repository: InterfaceRepository by lazy {
        Repository(retrofitService)
    }
}
```

RYSUNEK 4.28: Klient API *TravelAppRetrofitClient*

Klasa *Repository* odpowiedzialna jest za wywoływanie odpowiednich metod z serwisu *TravelAppApiService* i przekazywanie im niezbędnych danych, które serwer później wykorzysta. Repozytorium odpowiada za przetworzenie odpowiedzi od Retrofit na odpowiednie klasy danych. Klasa *Repository* wykorzystana jest przez modele widoku, które wywołują funkcje z *Repository* i przekazują im niezbędne dane potrzebne serwerowi np. id użytkownika, które zostanie wykorzystane przez serwer do pobrania z bazy danych informacji o użytkowniku o podanym id. Przykładowa metoda odpowiedzialna za obsługę metody *getUserProfileResult* z *TravelAppApiService* znajduje się na rys. 4.29.

```

override suspend fun getUserProfileResult(token: String?, id: Int): UserProfileResult? {
    val response: Response<UserProfileResult> =
        travelAppApiService.getUserProfileResult(
            token = "Bearer $token",
            id = id
        )
    val userProfileResult: UserProfileResult?
    if (response.isSuccessful) {
        userProfileResult = response.body()
        userProfileResult?.resultCode = response.code()
        return userProfileResult
    } else {
        val errorBody = response.errorBody()?.string()
        userProfileResult = Moshi.Builder().build().adapter(UserProfileResult::class.java)
            .fromJson(errorBody!!)
        userProfileResult?.resultCode = response.code()
        return userProfileResult
    }
}

```

RYSUNEK 4.29: Metoda *getUserProfileResult* z *Repository*

Metoda *getUserProfileResult* z rys. 4.29 odpowiedzialna jest za pobranie danych użytkownika z API i przetworzenie odpowiedzi od Retrofit na klasę danych *UserProfileResult* przedstawioną na rys. 4.30. Metoda wywołuje *getUserProfileResult* z serwisu *travelAppApiService* i przekazuje jej token użytkownika i id niezbędne do pobrania informacji o konkretnym użytkowniku z bazy danych. Następnie odpowiedź z API jest przetwarzana na klasę danych *UserProfileResult*, która jest zwracana do modelu widoku, który wywołał metodę z *Repository*. Funkcja *getUserProfileResult* z *Repository* sprawdza, czy komunikacja z wykorzystaniem HTTP zakończyła się sukcesem na podstawie kodu odpowiedzi. Odpowiedni model widoku przetwarza informacje takie jak kod odpowiedzi, nazwę użytkownika, opis, preferencje podróżnicze i pozostałe elementy składowe profilu i decyduje na podstawie kodu odpowiedzi, czy wyświetlić profil z danymi, czy informację o błędzie związanym z pobieraniem profilu.

```

@JsonClass(generateAdapter = true)
data class UserProfileResult(
    var resultCode: Int?,
    @Json(name = "avatar")
    val avatar: String?,
    @Json(name = "bio")
    val bio: String?,
    @Json(name = "facebook_link")
    val facebookLink: String?,
    @Json(name = "id")
    val id: Int?,
    @Json(name = "instagram_link")
    val instagramLink: String?,
    @Json(name = "message")
    val message: String?,
    @Json(name = "name")
    val name: String?,
    @Json(name = "planned_travels_count")
    val plannedTripsCount: Int?,
    @Json(name = "travel_preferences")
    val travelPreferences: List<TravelPreference?>?,
    @Json(name = "finished_travels_count")
    val tripsCount: Int?,
    @Json(name = "x_link")
    val xLink: String?
)

```

RYSUNEK 4.30: Klasa danych *UserProfileResult* zawierająca informacje o pobranym profilu z bazy danych

4.1.4 Integracja aplikacji z mapami

Integracja z mapami jest kluczową częścią aplikacji. Mapy dodane do TravelApp umożliwiają użytkownikom wizualizację swoich podróży oraz przeglądanie szczegółów podróży z poziomu interaktywnej mapy. Dodane do aplikacji mapy pozwalają na:

- Wyszukiwanie lokalizacji, punktu na mapie na podstawie wpisanego adresu w wyszukiwarce lokalizacji;
- Manualny wybór lokalizacji na mapie za pomocą kliknięcia na mapę w wybrane miejsce;
- Pobieranie map statycznych do podglądu lokalizacji w formularzu podróży (rys. 4.21b) i liście miejsc;
- Wyświetlanie podróży na mapie w postaci znaczników, które zawierają dodatkowe informacje o danym miejscu - rys. 4.19a.

W celu integracji aplikacji z mapami zostało założone konto na stronie mapbox.com. Następnym krokiem była konfiguracja klucza publicznego w projekcie w lokalizacji *app/res/values*. W tym celu został stworzony plik *mapbox_access_token.xml* (rys. 4.31).

```
<?xml version="1.0" encoding="utf-8"?>
<resources xmlns:tools="http://schemas.android.com/tools">
  <string name="mapbox_access_token" translatable="false" tools:ignore="UnusedResources">YOUR_MAPBOX_ACCESS_TOKEN</string>
</resources>
```

RYSUNEK 4.31: Konfiguracja klucza publicznego w projekcie

Następnie w Gradle Scripts w pliku *settings.gradle.kts* należy odpowiednio skonfigurować *dependencyResolutionManagement*. Ten krok wymaga podania nazwy użytkownika oraz hasła w celu dostępu do repozytorium Mapboxa. Więcej szczegółów dotyczących całej integracji aplikacji z mapami znajduje się na stronie dokumentacji Mapboxa [18].

Kolejnym krokiem była odpowiednia konfiguracja zależności w pliku *build.gradle.kts* przedstawiona na rys. 4.32.

```
implementation(libs.maps.compose)
implementation(libs.mapbox.search.android)
implementation(libs.mapbox.search.android.ui)
implementation(libs.mapbox.search.android)
```

RYSUNEK 4.32: Konfiguracja zależności niezbędnych do działania map od Mapbox

Ostatnim krokiem integracji z mapami od Mapbox jest implementacja komunikacji z API Mapboxa. Ten krok obejmuje wykorzystanie Retrofit do komunikacji z API Mapbox oraz obsługę odpowiedzi przy użyciu Moshi. W celu pełnego zintegrowania aplikacji z interaktywnymi mapami został zaimplementowany serwis *MapboxApiService*, którego fragment z funkcją *searchPlace* przedstawia rys. 4.33. Dodanie klienta API Mapboxa do aplikacji pozwoliło na implementację wyszukiwania lokalizacji za pomocą wpisanego adresu i pobieranie map statycznych wykorzystanych np. w podglądzie dodanej lokalizacji w formularzu.

```
interface MapboxApiService {
    @GET("geocoding/v5/mapbox.places/{place}.json")
    suspend fun searchPlace(
        @Path("place") place: String,
        @Query("access_token") accessToken: String,
        @Query("limit") limit: Int = 1
    ): Response<GeocodingSearchResponse>
}
```

RYSUNEK 4.33: *MapboxApiService*

Podobnie jak w integracji z API, w tej części również zaimplementowano klienta *MapboxClient* (rys. 4.34), który zawiera adres API Mapboxa i konfiguruje klienta Retrofit. W odróżnieniu do *TravelAppRetrofitClient*, *MapboxClient* nie przekierowuje serwisu *MapboxApiService* do repozytorium.

```
class MapboxClient {
    private val url = "https://api.mapbox.com/"
    private val moshi: Moshi = Moshi.Builder().add(KotlinJsonAdapterFactory()).build()
    private val retrofit: Retrofit = Retrofit.Builder()
        .baseUrl(url)
        .addConverterFactory(MoshiConverterFactory.create(moshi))
        .build()

    val mapboxService: MapboxApiService =
        retrofit.create(MapboxApiService::class.java)
}
```

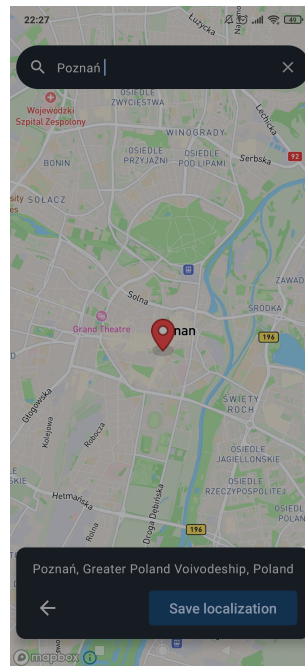
RYSUNEK 4.34: *MapboxClient*

Modele widoku takie jak *SearchMapViewModel* wykorzystują *MapboxClient* do wywoływania odpowiednich metod do komunikacji z API Mapboxa, pozwalających na wyszukiwanie miejsc, pobieranie map statycznych, pobieranie adresu lokalizacji.

Podstawową metodą pozwalającą użytkownikowi na wyszukiwanie lokalizacji na podstawie wpisanego w wyszukiwarce adresu jest *searchPlace* (rys. 4.33), która wykorzystuje punkt końcowy *geocoding/v5/mapbox.places/place.json* do pobrania odpowiedzi *GeocodingSearchResponse* za pomocą HTTP GET z API Mapboxa, która zawiera adres wyszukanej lokalizacji oraz współrzędne niezbędne do ustawienia punktu na mapie. Metoda pobiera od użytkownika następujące informacje:

- Adres szukanego miejsca, przekazywana po ścieżce URL za pomocą *@PATH("place")*.
- Token wykorzystany do autoryzacji, przekazany za pomocą *@QUERY("access_token")*.
- Ilość zwróconych wyników, w tym przypadku ograniczono liczbę wyników do jednego. Parametr przekazany za pomocą *@QUERY("limit")*.

Mapa z wyszukiwarką przedstawiona jest na rys. 4.35, składa się z ekranu *SearchMapScreen* i modelu widoku o nazwie *SearchMapViewModel*. *SearchMapScreen* wyświetla mapę, wyszukiwarkę oraz wyszukany adres w dolnej części ekranu. Wyszukiwarka umożliwia użytkownikowi wpisanie adresu, a po kliknięciu przycisku lupy na klawiaturze urządzenia wywołuje z *SearchMapViewModel* metodę *searchPlace* i przekazuje jej argument *query* czyli adres wpisany przez użytkownika oraz token. Metoda *searchPlace* wykorzystuje z serwisu *MapboxApiService* metodę o tej samej nazwie i przekazuje jej szukany adres i token (rys. 4.36), następnie Retrofit komunikuje się z API Mapboxa w celu uzyskania odpowiedniego wyniku. Odpowiedź zwracana przez Retrofit dla metody *searchPlace* przekształcana jest na klasę danych *GeocodingSearchResponse* przedstawioną na rys. 4.37. Klasa zawiera istotne dane takie jak współrzędne i adres wyszukanego miejsca potrzebne przy automatycznym zaznaczaniu punktu na mapie oraz przy wyświetlaniu adresu wyszukanej lokalizacji, są to informacje istotne dla użytkownika podczas wyszukiwania lokalizacji na mapie.

RYSUNEK 4.35: Mapa z wyszukiwarką - *SearchMapScreen*

```
fun searchPlace(place: String, accessToken: String) {
    viewModelScope.launch {
        try {
            if (place != "") {
                val response = mapboxClient.mapboxService.searchPlace(
                    place = place,
                    accessToken = accessToken
                )
                if (response.isSuccessful) {
                    val responseBody = response.body()
                    if (responseBody?.features?.isNotEmpty() == true) {
                        val coordinates = responseBody.features.first().geometry.coordinates
                        point = Point.fromLngLat(coordinates[0], coordinates[1])
                        placeName = responseBody.features.first().placeName
                    }
                }
            }
        }
    }
}
```

RYSUNEK 4.36: Metoda *searchPlace* z modelu widoku

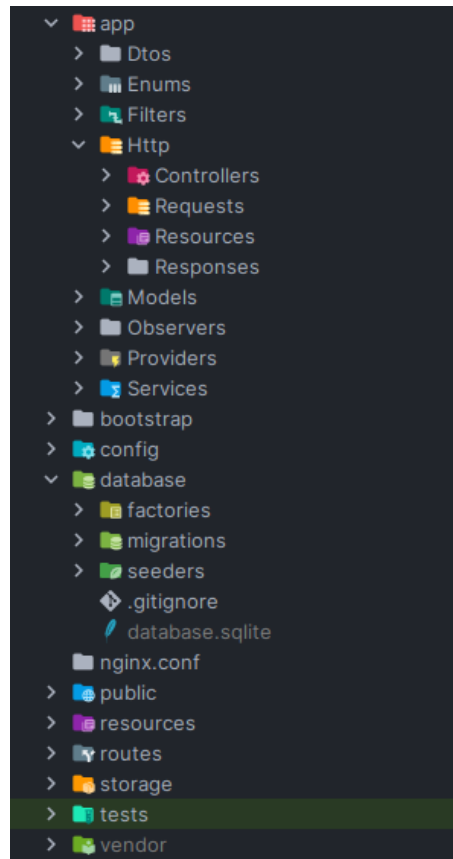
```
@JsonClass(generateAdapter = true)
data class GeocodingSearchResponse(
    @Json(name = "query") val query: List<String>,
    @Json(name = "features") val features: List<Feature>
) {
    @JsonClass(generateAdapter = true)
    data class Feature(
        @Json(name = "place_name") val placeName: String,
        @Json(name = "geometry") val geometry: Geometry
    ) {
        @JsonClass(generateAdapter = true)
        data class Geometry(
            @Json(name = "coordinates") val coordinates: List<Double>
        )
    }
}
```

RYSUNEK 4.37: *GeocodingSearchResponse*

4.2 Implementacja serwera z API

4.2.1 Struktura projektu

Struktura projektu API została zorganizowana zgodnie z najlepszymi praktykami, tak aby zapewnić czytelność kodu oraz łatwe skalowanie aplikacji. Struktura katalogów w projekcie została przedstawiona na rys. 4.38.



RYSUNEK 4.38: Struktura projektu API

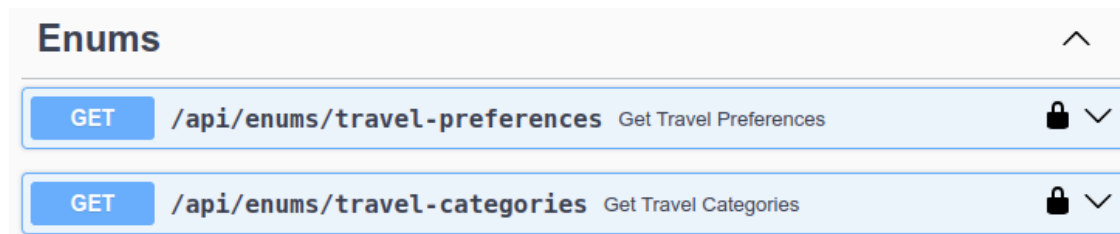
- **app** - główny katalog zawierający logikę aplikacji;
 - **Dtos (Data Transfer Objects)** - katalog zawiera obiekty transferu danych przenoszące dane między różnymi warstwami aplikacji;
 - **Enums** - folder zawiera enumeracje takie jak *FriendStatus*, wykorzystywane jako stałe wartości;
 - **Filters** - katalog zawierający klasy filtrów, na przykład *TimeLineIndexFilter*, który jest wykorzystywany przy filtrowaniu podróży na stronie głównej aplikacji;
 - **Http** - odpowiada za warstwę komunikacji HTTP;
 - * **Controllers** - katalog zawierający kontrolery obsługujące przetwarzanie żądań;
 - * **Requests** - folder zawierający klasy walidujące żądania HTTP;
 - * **Resources** - katalog zawiera szablony odpowiedzi API;
 - * **Responses** - folder zawiera klasy zasobów, odpowiedzialne za formatowanie odpowiedzi HTTP;
- **bootstrap** - folder zawierający pliki inicjalizujące aplikację;

- **config** - folder zawiera pliki konfiguracyjne, z podziałem na poszczególne komponenty;
- **database** - folder, który odpowiada za warstwę bazy danych;
 - **factories** - zawiera klasy fabryk, generujących dane wykorzystywane w testach;
 - **migrations** - folder zawierający migracje, wykorzystywane do modyfikowania struktury bazy danych;
 - **seeders** - katalog zawierający klasy wprowadzające początkowe dane do bazy danych;
- **public** - katalog zawierający publicznie dostępne pliki, takie jak plik *index.php*, który jest punktem wejściowym do aplikacji;
- **resources** - folder zawierający zasoby aplikacji, takie jak pliki językowe czy widoki;
- **routes** - folder zawiera pliki opisujące kierowanie żądań HTTP do odpowiednich kontrolerów;
- **storage** - katalog zawierający pliki wgrywane przez użytkowników aplikacji, w przypadku konfiguracji lokalnego magazynu, oraz logi generowane przez aplikację;
- **tests** - folder zawiera testy jednostkowe i funkcjonalne;
- **vendor** - folder zawierający zależności aplikacji, którymi zarządza Composer.

4.2.2 Opis punktów końcowych API

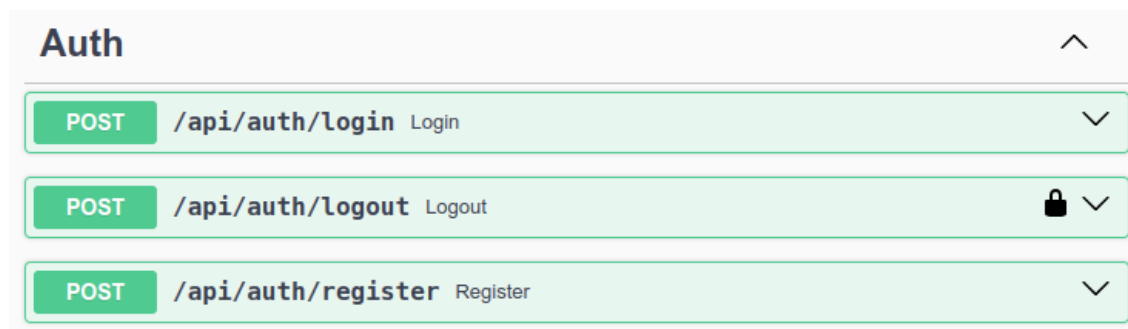
Najprostsze punkty końcowe są wykorzystywane w aplikacji mobilnej do dynamicznego pobierania danych do list wielokrotnego wyboru, dzięki czemu w przypadku dodania nowych kategorii miejsc lub preferencji podróźniczych, zamiast osobnego dodawania w jednej i drugiej aplikacji, dodanie jest konieczne tylko w bazie danych, ponieważ przy każdym otwarciu formularza aplikacja pobiera te dane z serwera. Rys. 4.39 przedstawia listę tych punktów końcowych, ukazuje metodę HTTP, url oraz nazwę. W przypadku tych, do których dostęp powinni mieć tylko użytkownicy zalogowani, po prawej stronie widnieje ikona kłódki. Wtedy aplikacja wraz z żądaniem musi przysłać w nagłówku żądania HTTP Bearer token wykorzystywany do autoryzacji, w przeciwnym razie aplikacja zwróci kod odpowiedzi 401, wraz z informacją, że użytkownik jest niezalogowany.

- **/api/enums/travel-preferences** - zwraca listę możliwych preferencji do wyboru w profilu użytkownika;
- **/api/enums/travel-categories** - zwraca listę kategorii dla miejsc w tworzeniu i edycji podróży.



RYСУNEK 4.39: Punkty końcowe - moduł z danymi do list opcji wyboru

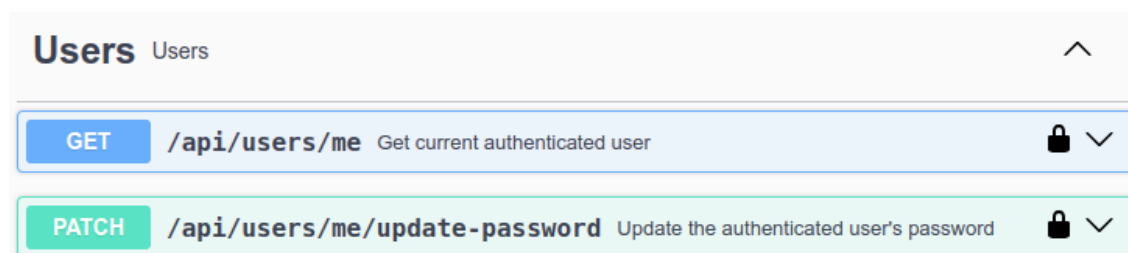
Kolejne punkty końcowe przedstawione na rys. 4.40, zostały zaimplementowane z myślą o autoryzacji użytkowników. Moduł ten odpowiada za rejestrację kont nowych użytkowników, logowanie użytkowników posiadających już zarejestrowane konto, oraz wylogowywanie.



RYSUNEK 4.40: Punkty końcowe -moduł autoryzacji

- **/api/auth/login** - jest odpowiedzialny za logowanie i przyjmuje następujące parametry: e-mail, hasło oraz nazwę urządzenia. W przypadku wprowadzenia przez użytkownika poprawnych danych, serwer zwraca kod odpowiedzi 200 oraz Bearer Token, który jest później wykorzystywany przez aplikację mobilną w celu autoryzacji dostępu do zabezpieczonych punktów końcowych. W przypadku przesłania pustego pola serwer zwróci kod odpowiedzi 422 wraz z informacją o konieczności wypełnienia danego pola. Gdy użytkownik prześle błędne dane, serwer zwróci kod błędu 401, wraz z informacją, że przesłane dane są niepoprawne;
- **/api/auth/logout** - wylogowywanie polega na usunięciu aktualnie wykorzystywanego tokenu przez użytkownika z bazy danych, przez co użytkownik za jego pomocą nie będzie mógł już wykonać żadnej akcji wymagającej autoryzacji. Pomyślne wylogowanie zwraca kod odpowiedzi 200;
- **/api/auth/register** - podczas rejestracji użytkownik przesyła nazwę użytkownika, adres e-mail, który musi być unikalny, oraz hasło. Jeżeli dane będą poprawne, serwer zwróci kod 200 oraz Bearer Token, w innym przypadku zwróci kod odpowiedzi 422, wraz z informacją o błędach w danych polach.

Jednym z prostszych modułów jest moduł użytkowników przedstawiony na rys. 4.41.



RYSUNEK 4.41: Punkty końcowe - moduł użytkowników

- **/api/users/me** - pozwala pobrać aplikacji podstawowe dane zalogowanego użytkownika, które zostaną wyświetlone w interfejsie, takie jak adres URL do zdjęcia profilowego, nazwa użytkownika, jego id oraz e-mail;
- **/api/users/me/update-password** - umożliwia zmienić hasło po uprzednim wpisaniu poprawnego poprzedniego hasła. W innym przypadku zmiana hasła nie powiedzie się.

Kolejny moduł powiązany z użytkownikami jest odpowiedzialny za ich profile, przedstawiony na rys. 4.42.

Profiles			^
GET	/api/profiles/{user}	Get user's profile	🔒 ✓
POST	/api/profiles/{user}/update	Update user's profile	🔒 ✓
GET	/api/profiles/search	Search profiles	🔒 ✓

RYSUNEK 4.42: Punkty końcowe - moduł profili użytkowników

- **/api/profiles/{user}** - umożliwia pobranie informacji o profilu danego użytkownika, gdzie *user* to parametr przyjmujący *id* użytkownika;
- **/api/profiles/{user}/update** - służy do aktualizacji profilu użytkownika, należy przekazać *id* użytkownika, pozwala na aktualizowanie nazwy użytkownika adresu e-mail, zdjęcia profilowego, preferencji podróżniczych oraz linków do innych portali społecznościowych. W odpowiedzi zwraca zaktualizowane dane profilu użytkownika;
- **/api/profiles/search** - jest wykorzystywany w wyszukiwarce profili, za pomocą wpisanej nazwy umożliwia wyszukiwanie profili.

Jeden z większych modułów to moduł zarządzania znajomymi, przedstawiony na rys. 4.43. To dzięki niemu jest to aplikacja społecznościowa i może łączyć użytkowników oraz ich podróże.

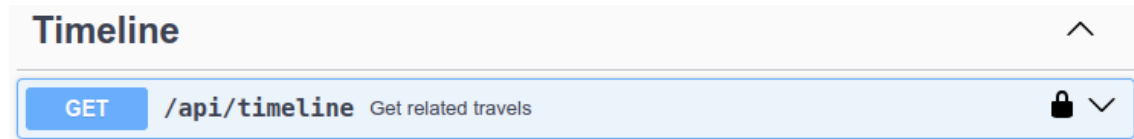
Friends			^
GET	/api/friends/list	Get a list of users friends	🔒 ✓
GET	/api/friends/requests	Get pending friend requests	🔒 ✓
POST	/api/friends/send-request	Send friend request	🔒 ✓
PATCH	/api/friends/accept-request/{friend_request}	Accept friend request	🔒 ✓
DELETE	/api/friends/reject-request/{friend_request}	Reject friend request	🔒 ✓
DELETE	/api/friends/delete/{friend_id}	Delete friend	🔒 ✓

RYSUNEK 4.43: Punkty końcowe - moduł znajomych

- **/api/friends/list** - zwraca listę znajomych użytkownika;
- **/api/friends/requests** - zwraca listę oczekujących zaproszeń do znajomych;
- **/api/friends/send-request** - służy do wysłania zaproszenia do grona znajomych, należy przekazać parametr jakim jest *id* użytkownika, do którego zaproszenie ma zostać wysłane;
- **/api/friends/accept-request/{friend_request}** - akceptuje otrzymane zaproszenie do znajomych, paramentr *friend_request* to *id* zaproszenia do znajomych;

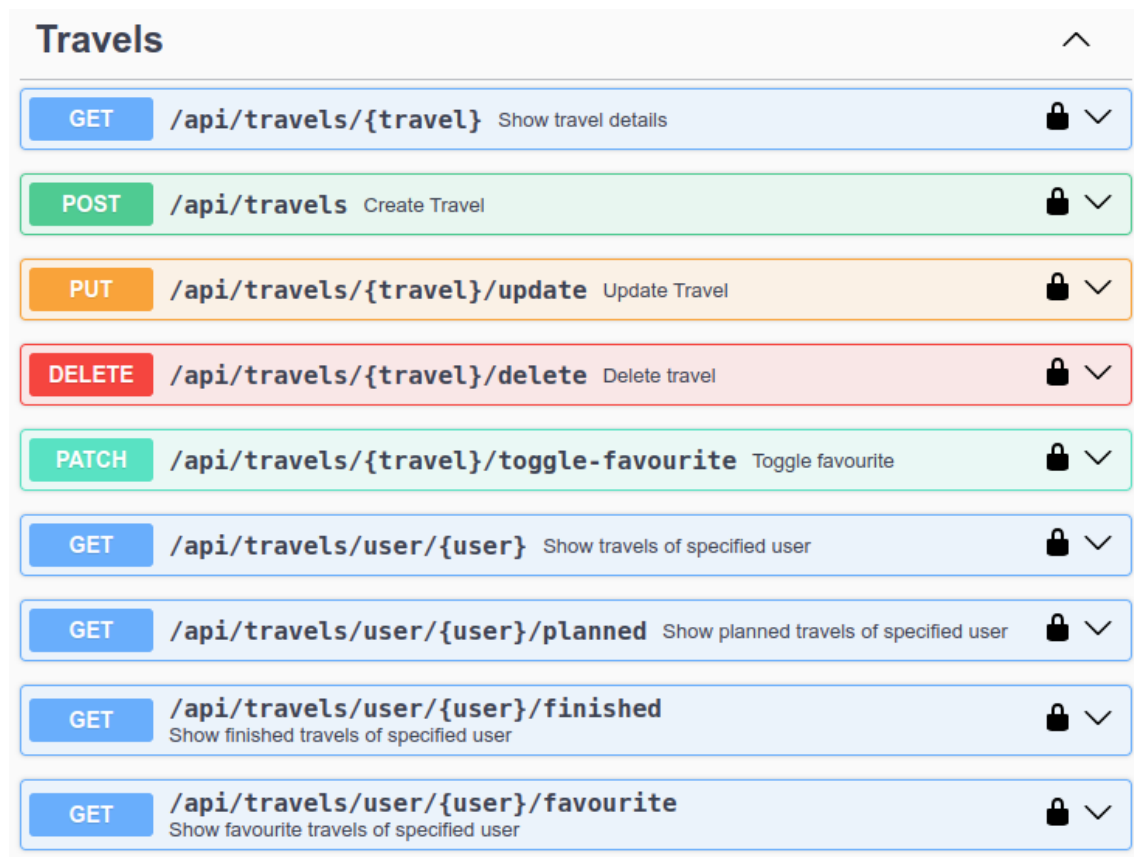
- **/api/friends/reject-request/{friend_request}** - odrzuca zaproszenie do znajomych o podanym *id*;
- **/api/friends/delete/{friend_id}** - usuwa znajomego o podanym *id* z listy znajomych.

Punkt końcowy **/api/timeline**, to dzięki niemu użytkownik po wejściu w aplikację na stronie głównej widzi najnowsze podróże swoich znajomych. Pobiera on listę podróży wszystkich znajomych użytkownika, posortowane od najnowszych. Został przedstawiony na rys. 4.44.



RYSUNEK 4.44: Punkty końcowe - moduł osi czasu

Największym modułem, zawierającym największą liczbę punktów końcowych, jest moduł podróży przedstawiony na rys. 4.45.

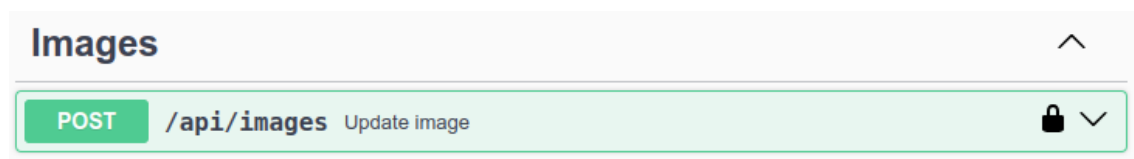


RYSUNEK 4.45: Punkty końcowe - moduł podróży

- **/api/travels/{travel}** - podaniu *id* podróży, zwraca jej szczegóły wraz z miejscami oraz adresami URL zdjęć,
- **/api/travels** - służy do tworzenia nowej podróży, przyjmuje dane podróży oraz przypisane miejsca,
- **/api/travels/{travel}/update** - umożliwia właścicielowi edycje istniejącej podróży o podanym *id*,

- `/api/travels/{travel}/delete` - usuwa podróż o podanym *id*,
- `/api/travels/{travel}/toggle-favourite` - dodaje lub usuwa podróż z listy ulubionych,
- `/api/travels/user/{user}` - zwraca uproszczoną listę podróży danego użytkownika, którego *id* zostało przesłane. Lista ta nie zawiera danych o miejscach podróży.
- `/api/travels/user/{user}/planned` - zwraca uproszczoną listę zaplanowanych podróży;
- `/api/travels/user/{user}/finished` - zwraca listę ukończonych podróży;
- `/api/travels/user/{user}/favourite` - zwraca listę podróży oznaczonych jako ulubione.

Punkt końcowy przedstawiony na rys. 4.46. `/api/images` pozwala użytkownikowi przesłać zdjęcie, jest on uniwersalny, ponieważ to po stronie aplikacji mobilnej dodawane są parametry, gdzie ma zostać przypisane zdjęcie.



RYSUNEK 4.46: Punkty końcowe - moduł przesyłania zdjęć

4.2.3 Opis implementacji punktu końcowego Timeline

Punkt końcowy `/api/timeline`, służący do pobierania danych wykorzystywanych na osi czasu, posłuży do przedstawienia procesu obsługi żądania HTTP po stronie serwera z API. Poniżej przedstawiono szczegółowy przebieg obsługi żądania od momentu jego przyjęcia przez serwer, aż po zwrócenie odpowiedzi do klienta. Opisany sposób implementacji pozwala na łatwą rozbudowę aplikacji i utrzymanie jej w przyszłości.

1. Definicja trasy (Route)

Kod w pliku `routes/api.php` definiuje trasę dla punktu dostępowego `/api/timeline`, przedstawia to rys. 4.47. Trasa jest zabezpieczona za pomocą middleware `auth:sanctum`, który wymaga, aby użytkownik był uwierzytelniony wykorzystując Bearer Token. Żądanie jest przekierowane do klasy `TimelineController`, a dokładnie do metody `index()`.

```
Route::middleware(['auth:sanctum', 'api'])->group(function () {
    Route::get('uri: '/timeline', [TimelineController::class, 'index']);
});
```

RYSUNEK 4.47: Kod serwerowego API - plik `routes/api.php`

2. Walidacja żądania (TimelineIndexRequest)

Do kontrolera trafia obiekt klasy `TimelineIndexRequest` przedstawiony na rys. 4.48. W tej klasie zdefiniowane są reguły walidacji dla parametrów: `page`, `date_from`, `date_to` oraz `sort_direction`. Wszystkie te parametry są opcjonalne (oznaczenie *nullable*). Parametr `page` musi być liczbą typu integer, a parametry `date_from` oraz `date_to` muszą być datami, natomiast dla parametru `sort_direction` może być przesłana tylko wartość `asc` lub `desc`.

```
class TimelineIndexRequest extends FormRequest
{
    ⚡ Rafal Grudziak
    public function rules(): array
    {
        return [
            'page' => ['nullable', 'integer'],
            'date_from' => ['nullable', 'date'],
            'date_to' => ['nullable', 'date'],
            'sort_direction' => ['nullable', 'in:asc,desc'],
        ];
    }
}
```

RYSUNEK 4.48: Kod serwerowego API - plik TimeLineIndexRequest.php

3. Kontroler (TimelineController)

Kontroler przyjmuje obiekt żądania *TimelineIndexRequest* i serwis *TimelineService* poprzez wstrzykiwanie zależności, co przedstawia rys. 4.49. Żądanie jest walidowane, a dane są przekazywane do obiektu transferu danych *TimelineIndexDto*. Następnie wywoływana jest metoda *filter()* z serwisu *TimeLineService*, w której filtrowane są dane na podstawie przekazanych filtrów. Wynik następnie jest paginowany i przekształcany do formatu JSON za pomocą zasobu *TimelineResource*.

```
class TimelineController extends BaseController
{
    ⚡ Rafal Grudziak *
    public function index(TimelineIndexRequest $request, TimelineService $timelineService): JsonResponse
    {
        $dto = new TimelineIndexDTO(...$request->validated());
        return PaginatedResponse::format($timelineService->filter($dto)->paginate(perPage: 10),
            resourceClass: TimelineResource::class);
    }
}
```

RYSUNEK 4.49: Kod serwerowego API - plik TimeLineController.php

4. Obiekt transferu danych (TimelineIndexDto)

TimelineIndexDto to obiekt transferu danych, który służy do przenoszenia tylko potrzebnych danych z żądania HTTP do logiki biznesowej znajdującej się w serwisie. Kod przedstawiony jest na rys. 4.50. Na wypadek nie przekazania danych, które są opcjonalne, przypisana jest im wartość domyślna *null*.

```
readonly class TimelineIndexDto implements FilterDto
{
    1 usage ⚡ Rafal Grudziak
    public function __construct(
        public readonly int $page = 1,
        public readonly ?string $date_from = null,
        public readonly ?string $date_to = null,
        public readonly ?string $sort_direction = null,
    ) {}
}
```

RYSUNEK 4.50: Kod serwerowego API - plik TimeLineIndexDTO.php

5. Serwis (TimelineService)

Serwis *TimelineService*, który obsługuje logikę biznesową przedstawia rys. 4.51. Metoda *filter()* przyjmuje obiekt *TimelineIndexDto* i tworzy instancję *TimelineIndexFilter* wykorzystując *Query Builder* oraz dane przekazywane w *TimelineIndexDto*. Serwis obsługuje filtrowanie danych na podstawie kryteriów podanych w żądaniu.

```
class TimelineService extends ModelService
{
    2 usages  ⚡ Rafal Grudziak
    protected function getModelClass(): string
    {
        return Travel::class;
    }

    ⚡ Rafal Grudziak
    public function filter(TimelineIndexDto $dto): BaseIndexFilter
    {
        return app()->make([abstract: TimelineIndexFilter::class, [
            'builder' => $this->getModelQuery(),
            'filterDto' => $dto,
        ]]);
    }
}
```

RYSUNEK 4.51: Kod serwerowego API - plik TimeLineService.php

6. Filtr (TimelineIndexFilter)

TimelineIndexFilter jest klasą odpowiedzialną za stosowanie filtrów na zapytaniu bazy danych, jej implementacja została przedstawiona na rys. 4.52. Metoda *filter()* stosuje filtry, które w przypadku otrzymania wartości z żądania nakładają odpowiednie reguły:

- **filterOnlyFriendsTravels()** - filtruje podróże, aby pokazać tylko te, które zostały dodane przez znajomych zalogowanego użytkownika,
- **filterByDateRange()** - filtruje podróże na podstawie zakresu dat, podanych w żądaniu,
- **orderBy()** - sortuje wyniki według kolumny *created_at*, czyli daty dodania w kolejności określonej przez użytkownika w żądaniu (domyślnie malejąco).

```
class TimelineIndexFilter extends BaseIndexFilter
{
    ⚡ Rafal Grudziak *
    public function filter(): void
    {
        $this->filterOnlyFriendsTravels();
        $this->filterByDateRange();
        $this->orderBy( column: 'created_at', direction: $this->filterDto->sort_direction ?? 'desc');
    }

    1 usage  ⚡ Rafal Grudziak *
    protected function filterOnlyFriendsTravels(): void
    {
        $user = auth()->user();
        $friendIds = $user->friends()->pluck('id');
        $this->builder->whereIn( column: 'user_id', $friendIds);
    }

    1 usage  ⚡ Rafal Grudziak *
    protected function filterByDateRange(): void
    {
        if ($this->filterDto->date_from) {
            $this->builder->where( column: 'from', operator: '>=', $this->filterDto->date_from);
        }
        if ($this->filterDto->date_to) {
            $this->builder->where( column: 'to', operator: '<=', $this->filterDto->date_to);
        }
    }
}
```

RYSUNEK 4.52: Kod serwerowego API - plik TimeLineIndexFilter.php

7. Zasób (TimelineResource)

Zasób *TimelineResource* odpowiada za formatowanie danych przed ich zwróceniem do klienta. Kod tego zasobu został przedstawiony na rys. 4.53. Przy każdej podróży zasób formatuje jej dane, wykorzystując do tego *TravelListResource*, a także dane dotyczące właściciela podróży, które są formatowane za pomocą zasobu *UserResource* przedstawionego na rys. 4.54.

```
class TimelineResource extends JsonResource
{
    ⚡ Rafal Grudziak
    public function toArray(Request $request): array
    {
        return [
            'travel' => new TravelListResource($this->resource),
            'user' => new UserResource($this->resource->user),
        ];
    }
}
```

RYSUNEK 4.53: Kod serwerowego API - plik TimeLineResource.php

```
class UserResource extends JsonResource
{
    ⚡ Rafal Grudziak
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->resource->id,
            'email' => $this->resource->email,
            'name' => $this->resource->name,
            'avatar' => $this->resource->avatar ? Storage::url($this->resource->avatar) : null,
        ];
    }
}
```

RYSUNEK 4.54: Kod serwerowego API - plik UserResource.php

4.2.4 Bezpieczeństwo i autoryzacja

Bezpieczeństwo jest kluczowym aspektem aplikacji internetowych, zapewnia ochronę danych użytkowników oraz integralność systemu. Laravel dostarcza wiele gotowych narzędzi pomagających zapewnić bezpieczeństwo systemu, które zostały wykorzystane w API aplikacji TravelApp.

1. Autoryzacja i uwierzytelnianie

Do uwierzytelniania zastosowano zostało Laravel Sanctum, jest to rozwiązanie które podczas wysłania żądania HTTP do zalogowania użytkownika generuje token dostępowy, który jest zwracany do aplikacji mobilnej. Aplikacja przechowuje ten token i dodaje go do nagłówka każdego żądania wysłanego do punktu końcowego, który wykorzystuje middleware *auth:sanctum*. Middleware ten wymaga aby każde żądanie miało w nagłówku ważny Bearer Token, aby uzyskać dostęp do zasobu. Gdy token nie będzie umieszczony w nagłówku lub będzie nieważny, serwer zwróci kod błędu 401. Podczas wylogowywania aktualny token zostaje usunięty z bazy danych i za jego pomocą nie będzie możliwe uzyskanie dostępu do zasobu.

2. Walidacja danych wejściowych

W celu zapewnienia, że dane przekazywane do aplikacji są poprawne i zgodne z oczekiwaniami, oraz ochrony przed atakami stosowana jest walidacja. Laravel dostarcza wiele gotowych reguł do wykorzystania, a w przypadku gdy programista nie znajdzie gotowej, potrzebnej mu reguły, framework pozwala na stworzenie własnej w celu dostosowania do specyficz-

nych potrzeb aplikacji. Dodatkowo podczas stosowania walidacji, do kontrolera trafiają tylko dane, które są przesłane w polach zdefiniowanych przez programistę w regułach walidacji. Pozwala to uchronić aplikację przed różnorodnymi atakami. W przypadku wykrycia błędów walidacji, Laravel zwraca kod błędu 422 oraz plik JSON ze szczegółami błędów.

3. Szyfrowanie danych

Dane wrażliwe takie jak hasła, przed zapisaniem w bazie danych są szyfrowane. Laravel oferuje wbudowane narzędzia do szyfrowania danych, które zostały zastosowane w implementacji API. Podczas rejestracji użytkowników, dane użytkownika zapisywane są w bazie danych, przed zapisem stosowana jest metoda `Hash::make()`, której parametrem jest hasło, wykorzystuje algorytm bcrypt w celu zaszyfrowania hasła. Natomiast podczas logowania zastosowana została metoda `Auth::attempt()`, której parametrem jest tablica asocjacyjna zawierająca adres e-mail i hasło. Następnie Laravel sprawdza, czy taki użytkownik istnieje w bazie danych, jeżeli istnieje sprawdza czy hasło jest poprawne. W przypadku podania niepoprawnych danych, zwracany błąd celowo nie zawiera informacji, czy błąd wynika z braku istniejącego użytkownika zarejestrowanego za pomocą podanego adresu e-mail, czy hasło jest niepoprawne. Chroni to przed atakami typu brute-force.

4. Zabezpieczenie przed atakami typu SQL Injection

Ataki SQL Injection polegają na wstrzykiwaniu złośliwego kodu SQL do zapytań wykonywanych przez aplikację. Polega to na tym, że użytkownik zamiast nazwy użytkownika podczas wyszukiwania, może przesłać kod, który może spowodować zwrócenie mu danych do których nie powinien mieć dostępu. W celu ochrony przed takimi incydentami zastosowano dwie metody zapobiegania takim atakom:

- Pierwszą metodą, jest już wcześniej wspomniana walidacja, parametry są walidowane aby były określone przez programistę typu;
- Drugą metodą, jest zastosowany Query Builder z wiązaniem parametrów. Jest to narzędzie do tworzenia i wykonywania zapytań SQL. Polega na zastąpieniu wartości w zapytaniu SQL, specjalnymi symbolami zastępczymi, które są później zastępowane danymi wejściowymi. Dane te podczas podstawiania do zapytania, są traktowane jako zwykły tekst.

4.3 Baza danych

4.3.1 Projekt bazy danych

Tworzenie aplikacji w Laravelu nie wymaga projektowania bazy danych przed przystąpieniem do implementacji backendu. Zamiast pisania pełnego kodu SQL definiującego strukturę bazy danych, framework oferuje system migracji. Migracje to klasy PHP, definiujące schematy tabel w bazie danych, wykorzystując metody z frameworka Laravel. System ten umożliwia stopniowe tworzenie struktury bazy danych, bazujące na zdefiniowanych modelach.

Proces tworzenia tabeli:

1. **Definicja modelu** - w pierwszej kolejności tworzony jest model, który reprezentuje encję w bazie danych. Rys. 4.55 przedstawia zdefiniowany model reprezentujący miejsce.

- Pole **fillable** - tablica nazw kolumn w tabeli, z której aplikacja będzie korzystać;

- Metoda **travel()** - odzwierciedlenie relacji do encji *travels*, zwraca podróż, do której przypisana jest dana instancja miejsca;
- Metoda **category()** - odzwierciedla relację do encji *travel_categories* i zwraca kategorię do jakiej zostało przypisane to miejsce;
- Metoda **images()** - odzwierciedlenie relacji do encji *images*, zwraca obiekty klasy *Image* przypisane do danej instancji miejsca.

```

class Place extends Model
{
    use HasFactory;

    no usages
    protected $fillable = [
        'travel_id',
        'name',
        'description',
        'category_id',
        'longitude',
        'latitude'
    ];

    Rafal Grudziak
    public function travel(): BelongsTo
    {
        return $this->belongsTo([related: Travel::class]);
    }

    Rafal Grudziak
    public function category(): BelongsTo
    {
        return $this->belongsTo([related: TravelCategory::class]);
    }

    Rafal Grudziak
    public function images(): MorphMany
    {
        return $this->morphMany([related: Image::class, name: 'imageable']);
    }
}

```

RYSUNEK 4.55: Kod serwerowego API - model *Place*

2. **Napisanie migracji** - w pliku migracji przedstawionym na rys. 4.56, określa się szczegóły struktury tabeli, takie jak kolumny, typu danych i klucze obce.
 - Metoda **up()** - metoda ta jest wykorzystywana do utworzenia tabeli.
 - Metoda **down()** - jest ona wykorzystywana w sytuacji cofania migracji i usuwa tabelę.
3. **Wykonanie migracji** - po napisaniu migracji należy wykonać komendę: *sail artisan migrate* w głównym folderze repozytorium aplikacji, w celu wykonania migracji i utworzenia tabeli w bazie danych.

Rozdział 5

Testowanie aplikacji i API

5.1 Testowanie aplikacji

Testowanie oprogramowania odpowiada za weryfikację, czy działa ono zgodnie z wcześniej ustalonymi założeniami i funkcjonalnościami, oraz czy badany program jest wolny od błędów, które uniemożliwiają poprawne działanie. Testowanie aplikacji jest ważnym elementem tworzenia oprogramowania, ponieważ użytkownicy oczekują poprawnego działania aplikacji.

Testowanie aplikacji było istotnym etapem tworzenia TravelApp i przyczyniło się do identyfikacji i eliminacji błędów, które powstały podczas tworzenia aplikacji. Testowanie potwierdziło poprawne działanie aplikacji zgodne z założeniami systemu.

Zastosowane metody testowania aplikacji:

- **Testy automatyczne** - testy przeprowadzane za pomocą narzędzi, skryptów, które automatyzują proces testowania oprogramowania. Testowanie automatyczne pozwala na przeprowadzenie wielu testów w krótkim czasie [19].
- **Testy manualne** - testy wykonywane ręcznie przez testera, który sprawdza czy oprogramowanie zachowuje się zgodnie z ustalonymi wymaganiami. Testy manualne są bardziej czasochłonne w porównaniu z testami automatycznymi, ale pozwalają na prześledzenie działania funkcji i obserwację zachowania systemu oraz elementów UI [19].

Do wykonania testów automatycznych został wykorzystany framework JUnit4. Przykłady przeprowadzonych testów automatycznych znajdują się na rys. 5.1 i 5.3.

Test przedstawiony na rys. 5.1 sprawdza, czy elementy UI takie jak przycisk logowania, przycisk rejestracji, tekst oraz pola formularza logowania wyświetlają się poprawnie na ekranie logowania. Obiekty *LoginViewModel* i *NavController* są "mockowane" za pomocą biblioteki mockk. Mock to symulowany obiekt, który jest używany podczas przeprowadzania testów. Następnie *LoginScreen* ustawiany jest jako zawartość aktualnego ekranu za pomocą *composeTestRule.setContent*. Na końcu elementy UI zostają wyszukane za pomocą przypisanych do nich znaczników, funkcja *assertExists* sprawdza, czy dane komponenty UI istnieją. Wynik testu dla emulatora Pixel_8_API30 znajduje się na rys. 5.2.

```

@RunWith(AndroidJUnit4::class)
class LoginScreenTest {
    @get:Rule
    val composeTestRule = createComposeRule()

    @Test
    fun loginScreen_displaysAllMainComponents() {
        val viewModel = mockk<LoginViewModel>(relaxed = true)
        val navController = mockk<NavController>(relaxed = true)

        composeTestRule.setContent {
            LoginScreen(navController = navController, viewModel = viewModel)
        }

        composeTestRule.onNodeWithTag(testTag: "loginButton").assertExists()
        composeTestRule.onNodeWithTag(testTag: "createAccountButton").assertExists()
        composeTestRule.onNodeWithTag(testTag: "passwordInput").assertExists()
        composeTestRule.onNodeWithTag(testTag: "emailInput").assertExists()
        composeTestRule.onNodeWithTag(testTag: "loginText").assertExists()
    }
}

```

RYSUNEK 5.1: Test poprawnego wyświetlania elementów UI ekranu logowania

Status	1 passed	1 tests, 11 s 956 ms
Filter tests:	☒ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
Tests	Duration	Pixel_8_API_30
✓ Test Results	3 s	1/1
✓ LoginScreenTest	3 s	1/1
✓ loginScreen_displaysAllMainComponents	3 s	✓

RYSUNEK 5.2: Wynik testu sprawdzającego poprawne wyświetlanie elementów UI

Drugi przykładowy test przedstawiony na rys. 5.3 sprawdza, czy przycisk edycji profilu znajdujący się w ekranie profilu użytkownika przenosi do ekranu *EditProfileScreen* odpowiedzialnego za edycję profilu. Na przycisku zostaje wykonana akcja kliknięcia za pomocą *performClick*, a *verify* sprawdza, czy nawigacja do *EditProfileScreen* została wykonana. Wynik testu znajduje się na rys. 5.4.

Podsumowując, oba testy automatyczne zostały zaliczone, a elementy UI poprawnie się wyświetlają i spełniają wcześniej określone funkcje.

```

@Test
fun profileMainInfo_editProfileButtonNavigatesToEditScreen() {
    val navController = mockk<NavController>(relaxed = true)
    val viewModel = mockk<ProfileViewModel>(relaxed = true).apply {
        every { isOwnProfile } returns true
    }

    composeTestRule.setContent {
        ProfileMainInfoComponent(viewModel = viewModel, navController = navController)
    }

    val text = "Edit profile"

    composeTestRule.onNodeWithText(text).performClick()

    verify { navController.navigate(Screen.EditProfileScreen.destination) }
}

```

RYSUNEK 5.3: Test sprawdza czy przycisk związany z edycją profilu przenosi do ekranu edycji profilu

Status	1 passed	1 tests, 15 s 756 ms
Filter tests:	☒ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
Tests	Duration	Pixel_8_API_30
✓ Test Results	3 s	1/1
✓ ProfileScreenTest	3 s	1/1
✓ profileMainInfo_editProfileButtonNavigatesToEditScrei	3 s	✓

RYSUNEK 5.4: Wynik testu sprawdzającego działanie przycisku "Edit profile"

Testy manualne aplikacji polegały na ręcznym sprawdzaniu działania aplikacji, zostały sprawdzone różne scenariusze działania użytkownika od poprawnego zalogowania do aplikacji po usuwanie podróży z listy. Testy manualne pozwoliły na sprawdzenie poprawności działania aplikacji, zgodności z określonymi założeniami aplikacji, płynności działania aplikacji, wyglądu i zachowania elementów UI poszczególnych modułów.

Testy manualne obejmowały:

- **Pojedyncze funkcjonalności** - działanie przycisków - czy użytkownik zostanie zarejestrowany po kliknięciu przycisku rejestracji. Wyświetlanie elementów UI - czy wszystkie zaimplementowane elementy UI zostaną poprawnie wyświetlone na ekranie urządzenia;
- **Komunikacja z API** - pobieranie danych przez aplikację i sprawdzenie ich zgodności z danymi znajdującymi się w bazie danych serwera;
- **Złożone scenariusze** - możliwe scenariusze, które mogą wystąpić podczas użytkowania aplikacji. Scenariusze obejmują wiele powiązanych ze sobą akcji użytkownika w aplikacji np. założenie konta i pierwsza edycja profilu.

5.2 Testowanie API

Testowanie API jest bardzo ważnym procesem podczas tworzenia aplikacji, zapewnia poprawność działania, zgodność z dokumentacją oraz niezawodność nawet w specyficznych scenariuszach. Laravel udostępnia wiele narzędzi wspomagających testowanie aplikacji zarówno w środowisku lokalnym, jak i automatycznym. Aplikacja była testowana za pomocą testów jednostkowych, czyli testujących dane fragmenty kodu, wraz z wykorzystaniem "mocków" pozwalających na odizolowanie testowanego kodu od bazy danych czy innych klas.

Mock pozwala na weryfikowanie interakcji z innymi elementami aplikacji, programista w piśmym teście określa, jakie dokładnie metody oraz ile razy powinny zostać wykonane. Następnie testy sprawdzają, czy te interakcje faktycznie wystąpiły. Laravel umożliwia testowanie z wykorzystaniem mocków za pomocą narzędzia PHP Unit [20].

Test *testGetTravelPreferences* przedstawiony na rys. 5.5, sprawdza działanie metody *getTravelPreferences* kontrolera *EnumController*. Celem tego testu jest sprawdzenie, czy metoda *getTravelPreferences* działa prawidłowo i zwraca odpowiedni kod odpowiedzi HTTP. W tym teście tworzony jest mock klasy *Builder*, klasy odpowiedzialnej za budowanie zapytań do bazy danych. Klasa jest konfigurowana tak, aby metoda *get()* została wykonana dokładnie raz i zwróciła kolekcję. Następnie "mockowany" jest model *TravelPreference* i konfigurowany tak, aby została raz wywołana metoda *newQuery()* i zwróciła obiekt klasy *Builder*. W kolejnym kroku tworzona jest instancja klasy *EnumController* i wywoływana jest metoda *getTravelPreferences()*. Finalnie sprawdzane jest, czy kod odpowiedzi jest równy 200.

```
class EnumControllerTest extends TestCase
{
    /**
     * Rafal Grudziak *
     */
    public function testGetTravelPreferences(): void
    {
        $builder = $this->mock(abstract: Builder::class);
        $builder->shouldReceive(...methodNames: 'get')->once()->andReturn(collect());

        $this->mock(abstract: TravelPreference::class)
            ->shouldReceive(...methodNames: 'newQuery')->once()->andReturn($builder);

        $controller = app(abstract: EnumController::class);
        $response = $controller->getTravelPreferences();

        $this->assertSame(expected: 200, $response->getStatusCode());
    }
}
```

30ms Tests passed: 1 of 1 test - 30ms

RYSUNEK 5.5: Kod API - test jednostkowy testGetTravelPreferences

Celem testu *testStoreTravel* przedstawionego na rys. 5.6 jest sprawdzenie poprawności działania metody *storeTravel()* znajdującej się w serwisie *TravelService*. W pierwszej kolejności odbywa się "mockowanie" metody *Auth::id()*. Metoda ta odpowiada za zwrócenie *id* zalogowanego użytkownika, a więc w teście należy ją skonfigurować, aby zwróciła testowe *id*, w tym przypadku 123. W kolejnym kroku tworzony i wypełniany testowymi danymi jest obiekt transferu danych *TravelStoreDto*. Następnie "mockowane" i konfigurowane są klasy *Travel* oraz *Place*, aby finalnie wywołać

metodę `storeTravel()` z serwisu `TravelService` i sprawdzić czy poprawnie zwróciła instancję `Travel`. Dodatkowo test sprawdza poprawność ustawienia wszystkich atrybutów.

```
class TravelServiceTest extends TestCase
{
    public function testStoreTravel(): void
    {
        Auth::shouldReceive('id')
            ->once()
            ->andReturn(123);

        $dto = new TravelStoreDto([
            'name' => 'Test name',
            'from' => '01-01-2025',
            'to' => '01-10-2025',
            'longitude' => 52.396335,
            'latitude' => 16.955996,
            'places' => [
                [
                    'name' => 'Place name',
                    'category_id' => 1,
                    'longitude' => 52.396335,
                    'latitude' => 16.955996,
                ],
            ],
            'description' => 'Test description',
        ]);

        $travel = $this->mock(abstract: Travel::class);
        $travel->shouldReceive( ...methodNames: 'newInstance')->once()->andReturnSelf();
        $travel->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'name', $dto->name)->once()->andReturnSelf();
        $travel->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'description', $dto->description)->once()->andReturnSelf();
        $travel->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'from', $dto->from)->once()->andReturnSelf();
        $travel->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'to', $dto->to)->once()->andReturnSelf();
        $travel->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'longitude', $dto->longitude)->once()->andReturnSelf();
        $travel->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'latitude', $dto->latitude)->once()->andReturnSelf();
        $travel->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'user_id', 123)->once()->andReturnSelf();
        $travel->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'favourite', false)->once()->andReturnSelf();
        $travel->shouldReceive( ...methodNames: 'getAttribute')->with( ...args: 'id')->once()->andReturn(1);
        $travel->shouldReceive( ...methodNames: 'save')->once();

        $place = $this->mock(abstract: Place::class);
        $place->shouldReceive( ...methodNames: 'newInstance')->once()->andReturnSelf();
        $place->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'travel_id', 1)->once()->andReturnSelf();
        $place->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'category_id', $dto->places[0]->category_id)->once()->andReturnSelf();
        $place->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'name', $dto->places[0]->name)->once()->andReturnSelf();
        $place->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'description', $dto->places[0]->description)->once()->andReturnSelf();
        $place->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'longitude', $dto->places[0]->longitude)->once()->andReturnSelf();
        $place->shouldReceive( ...methodNames: 'setAttribute')->with( ...args: 'latitude', $dto->places[0]->latitude)->once()->andReturnSelf();
        $place->shouldReceive( ...methodNames: 'save')->once();

        $result = app(abstract: TravelService::class)->storeTravel($dto);
        $this->assertInstanceOf(expected: Travel::class, $result);
    }
}
```

Tests\Unit\Services > TravelServiceTest

29 ms Tests passed: 1 of 1 test - 29 ms

RYSUNEK 5.6: Kod API - test jednostkowy `testStoreTravel`

Rozdział 6

Wnioski i podsumowanie

6.1 Trudności napotkane w trakcie pracy

Realizacja projektu aplikacji TravelApp nie obyła się bez różnych trudności, które zostały napotkane w różnych fazach implementacji aplikacji. Problemy, które wystąpiły podczas tworzenia aplikacji, wymagały analizy, debugowania aplikacji i wprowadzania zmian w kodzie, co przyczyniło się do zwiększenia czasu tworzenia aplikacji TravelApp. W rozwiązywaniu problemów przydatne były narzędzia takie jak Logcat, Debugger i Profiler, które dostępne są w Android Studio.

Logcat to narzędzie do przeglądania zdarzeń, logów z działania aplikacji w konsoli. Za pomocą klasy *Log* programiści mogą wyświetlać wiadomości z informacjami z aplikacji w konsoli, przyspiesza to rozwiązywanie błędów aplikacji [21]. Klasa *Log* została wykorzystana przy wyświetlaniu odpowiedzi z serwera w różnych etapach ich przetwarzania od *Repository* do UI.

W początkowych fazach implementacji aplikacji jednym z problemów było niepoprawne przetwarzanie odpowiedzi z serwera na klasę danych, co skutkowało np. brakiem wiadomości (null w polu message) dla odpowiedzi serwera z kodem 401 lub 422. Problem polegał na niepoprawnym przypisaniu do zmiennej *loginResult* odpowiedzi *response.body()* nawet dla odpowiedzi z kodem 401 i 422. Treść zdeserializowanej odpowiedzi *response.body()* powinna zostać zapisana do zmiennej w sytuacji, gdy kod odpowiedzi znajduje się w zakresie od 200 do 299. Dla sytuacji, w których kod odpowiedzi HTTP należy do zakresu od 400 do 499 (błąd po stronie klienta) do zmiennej *loginResult* należy zapisać odpowiednio przekształcony *response.errorBody()*. Treść odpowiedzi *response.body()*, można wykorzystać dla przypadku, w którym HTTP zwraca odpowiedź z kodem 200-299 oznaczającym sukces. W przeciwnym wypadku należy wykorzystać zawarte w odpowiedzi *response.errorBody()*. Na rys. 6.1 przedstawione jest poprawna obsługa i przekształcanie odpowiedzi z serwera dla metody logowania w klasie *Repository*. Za pomocą narzędzia Logcat i Debuggera w Android Studio udało się ustalić, w którym miejscu w projekcie znajduje się błąd, a sam problem został szybko rozwiązany.

```

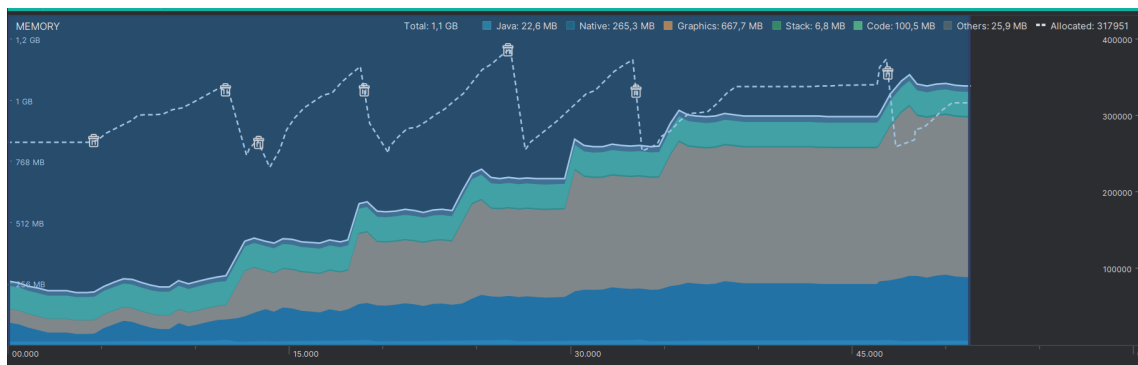
class Repository(private val travelAppApiService: TravelAppApiService) : InterfaceRepository {
    override suspend fun getLoginResult(request: LoginRequest): LoginResult? {
        val response: Response<LoginResult> = travelAppApiService.getLoginResult(request)
        val loginResult: LoginResult?

        if (response.isSuccessful) {
            loginResult = response.body()
            loginResult?.resultCode = response.code()
            return loginResult
        } else {
            val errorBody = response.errorBody()?.string()
            loginResult = Moshi.Builder().build().adapter(LoginResult::class.java)
                .fromJson(errorBody!!)
            loginResult?.resultCode = response.code()
            return loginResult
        }
    }
}

```

RYSUNEK 6.1: Metoda `getLoginResult` odpowiedzialna za przekształcanie odpowiedzi z API na klasę danych `LoginResult`

Najpoważniejszym problemem, który wystąpił podczas implementacji aplikacji, był wyciek pamięci związany z mapami. W celu sprawdzenia użycia pamięci wykorzystano narzędzie Profiler, które dało wgląd na stan aplikacji po kilku wejściach i wyjściach z ekranu wyświetlającego mapę podróży. Wyciek pamięci można zaobserwować na rys. 6.2, który przedstawia wykres użycia pamięci RAM urządzenia. Użycie pamięci rośnie z poziomu 256 MB na początku działania aplikacji do ponad 1,1 GB po wielokrotnym wchodzeniu i wychodzeniu z mapy - widoczne skoki użycia pamięci to momenty, w których użytkownik nawigował ponownie do ekranu z mapą. W krótkim czasie pamięć RAM urządzenia została szybko wyczerpana, co przełożyło się na zawieszenie i wyłączenie aplikacji.



RYSUNEK 6.2: Wykres z Profilera dotyczący użycia pamięci RAM

Problem wycieku związany był z niepoprawnym zarządzaniem nawigacją w aplikacji. Na rys. 4.35 widoczny jest dolny element UI odpowiedzialny za powrót do formularza podróży, wyświetlanie wyszukanego adresu oraz za zapis adresu w formularzu. Natomiast rys. 6.3 przedstawia implementację tego elementu. `NavController` w aplikacji odpowiedzialny jest za nawigację między ekranami, pozwala na przenoszenie się z jednego ekranu do drugiego, udostępnia do tego szereg różnych funkcji. W momencie zapisu lokalizacji (funkcja lambda `onSaveClick`) w wersji kodu z wyciekami pamięci wykorzystywała nawigację do formularza podróży za pomocą funkcji `navigate` z `NavControllera`. Funkcja `navigate` nawigowała do funkcji `@Composable` i jednocześnie dokładała do stosu nawigacyjnego kolejny element `@Composable`, do którego użytkownik mógł wrócić. Na rys. 6.3 zamiast funkcji `navigate` z `NavController` znajduje się `popBackStack`, który pozwala na powrót do poprzedniego ekranu i usunięcie bieżącego ekranu `@Composable` z stosu nawigacyj-

nego. W przypadku *navigate*, kolejne nawigacje do ekranu formularza zwiększały stos nawigacyjny, a brak usunięcia z stosu ekranu zawierającego mapę, czyli obiekt obciążający pamięć, powodował w krótkim czasie szybkie wyczerpanie pamięci.

Dodanie do funkcji *@Composable SearchMapScreen* kodu odpowiedzialnego za czyszczenie wykorzystywanych zasobów przez *MapView* i *PointAnnotationManager* oraz zamiana funkcji *navigate* z *NavController* na *popBackStack* rozwiązała problem wycieku pamięci. W rozwiązaniu tego problemu pomogło narzędzie Profiler, które umożliwiło podgląd zużycia pamięci przez aplikację w czasie rzeczywistym.

```
BottomPlaceCard(  
    placeName = viewModel.placeName,  
    onSaveClick = {  
        if (viewModel.point != null) {  
            sharedTravelViewModel.setNewPointInTravelOrPlace(viewModel.point!!)  
            navController.popBackStack()  
        }  
    },  
    onBackClick = { navController.popBackStack() },  
    modifier = Modifier  
        .align(Alignment.BottomCenter)  
        .padding(bottom = getDynamicPadding( parameter: 0.05f))  
        .padding(horizontal = 8.dp)  
)
```

RYSUNEK 6.3: Implementacja *BottomPlaceCard*

6.2 Perspektywy dalszego rozwoju aplikacji

Aplikacja *TravelApp* spełnia wszystkie wymagania projektowanego systemu i zawiera szereg funkcjonalności związanych z podróżami, mapami, które mogą zostać rozbudowane, a ponadto do aplikacji mogą zostać dodane nowe funkcjonalności rozszerzające jej aktualną wersję.

Pierwszym możliwym rozszerzeniem aplikacji jest wyświetlanie wielu zdjęć z przebiegu wyjazdu. API umożliwia dodanie kilku fotografii do jednej podróży, a także zwraca dla niej listę adresów URL, które pozwalają zlokalizować konkretne zdjęcia na serwerze. Wdrożenie funkcjonalności związanej z galerią zdjęć podróży, pozwoliłoby użytkownikom dodawać wiele wartych uwagi grafik i nie musieliby ograniczać się do jednej konkretnej. Przedstawiona funkcjonalność mogłaby istnieć w postaci prostej galerii zdjęć w ramach podróży, którą użytkownik mógłby przeglądać.

Kolejną funkcjonalnością, którą można dodać do aplikacji, jest wyświetlanie wszystkich podróży użytkownika na mapie. Użytkownik z wyświetlanego profilu mógłby przejść do mapy, która zawiera wszystkie podróże w formie znaczników z nazwą podróży. Kliknięcie w znacznik pozwoliłoby na wyświetlenie ogólnych informacji o podróży bez informacji o miejscach. Przedstawiona funkcjonalność uzupełniałaby informacje o liczbie podróży użytkownika, pozwalałaby obserwować wszystkie podróże na interaktywnej mapie oraz motywowałaby użytkowników do dalszego podróżowania po świecie. Realizacja przedstawionej funkcji polegałaby na pobraniu wszystkich podróży użytkownika, wybraniu współrzędnych podróży i rozmieszczeniu ich w formie znaczników na mapie. Dostęp do takiej mapy miałyby osoba wstawiająca podróże oraz jej znajomi - zabezpieczenie informacji.

Aktualnie użytkownik może wyszukać podróże wszystkich znajomych, posortować wyniki wyszukiwania po dacie podróży, ograniczyć wyniki wyszukiwań do określonej daty (rozpoczęcia i za-

kończenia). Dodanie nowych filtrów do wyszukiwarki umożliwiłoby dokładniejsze przeszukiwanie zbioru interesujących podróży znajomych w aplikacji TravelApp. Rozszerzenie obejmowałoby wyszukiwanie podróży po nazwie, filtrowanie po kraju, filtrowanie po dacie dodania, wyszukiwanie wpisu z wyjazdu konkretnego znajomego.

6.3 Ocena osiągniętych celów

Celem pracy było stworzenie aplikacji mobilnej TravelApp, przeznaczonej dla miłośników podróży, aplikacji umożliwiającej użytkownikom dokumentowanie odwiedzonych miejsc, planowanie przyszłych wycieczek oraz dzielenie się doświadczeniami z innymi użytkownikami. Funkcje przewidziane w TravelApp obejmowały zarządzanie profilami użytkowników, historie podróży z opisami i zdjęciami, planowanie wyjazdów oraz wyszukiwarkę wpisów z wycieczek i mapy z zaznaczonymi odwiedzonymi lokalizacjami. Ponadto aplikacja miała wspierać interakcje społecznościowe. Wszystkie cele założone przed rozpoczęciem tworzenia aplikacji zostały osiągnięte. Zaimplementowane funkcjonalności spełniają swoje założenia. Przyszli użytkownicy będą mogli planować i dokumentować swoje podróże, a także dzięki możliwości dodawania znajomych, użytkownicy mogą cieszyć się dzieleniem wspomnień ze swoimi znajomymi.

Warto zaznaczyć, że dokumentacja niniejszej pracy nie oddaje w pełni skali całego projektu. Implementacja aplikacji wymagała wiele godzin pracy i dużego zaangażowania. Dobrym przykładem do zobrazowania skali tego projektu jest liczba linii kodu. Implementacja serwera z API składa się z około 18 500 linii kodu, z czego pliki napisane własnoręcznie liczą ponad 4 400 linii. Większość pozostałych linii kodu pochodzi z pliku *composer.lock* (około 8 400 linii), który przechowuje szczegółowe informacje o wszystkich zależnościach wykorzystywanych w projekcie. Kolejne ponad 3 200 linii to plik z dokumentacją API, wygenerowany automatycznie na podstawie adnotacji napisanych w kontrolerach. Natomiast aplikacja mobilna liczy ponad 16 000 linii kodu. Świadczy to o ogromnym nakładzie pracy włożonym w stworzenie aplikacji.

6.4 Wnioski z realizacji projektu

Realizacja projektu wymagała współpracy zespołowej, zadania zostały podzielone tak, aby osoby odpowiedzialne za projekt mogły skupić się na swoim obszarze specjalizacji: jedna osoba była odpowiedzialna za aplikację mobilną, integrację z mapami Mapbox oraz integrację aplikacji z API, a druga za implementację serwerowego API i bazę danych. Realizacja projektu opierała się na architekturze klient-serwer, wymagało to wiedzy dotyczącej działania protokołu sieciowego HTTP. Implementacja projektu wymagała wiedzy związanej z różnymi technologiami oraz narzędziami wykorzystywanymi do tworzenia i wdrażania oprogramowania. Za pomocą narzędzi, technologii takich jak Laravel, Jetpack Compose, MySQL, Retrofit, Docker zrealizowano projekt zgodnie z wcześniej ustalonymi wymaganiami. Implementacja aplikacji wymagała ciągłego procesu nauki i poszerzania wiedzy z zakresu użytych technologii oraz narzędzi. Trudności towarzyszące podczas tworzenia systemu wymagały starannej analizy, przemyśleń i wyciągania odpowiednich wniosków w celu osiągnięcia założonych celów i optymalizacji działania systemu. Zadania związane z projektem zostały podzielone i zorganizowane w odpowiedniej kolejności, aby po każdym wykonanym zadaniu zintegrować funkcjonalność pomiędzy aplikacją na urządzenie mobilne i API. W celu ułatwienia organizacji pracy, została wykorzystana aplikacja ClickUp do zarządzania zadaniami [22]. Osiąganie postępów i celów związanych z aplikacją wymagało regularnych spotkań w celu weryfikacji postępów, planowania kolejnych prac i rozwiązywania problemów oraz trudności.

Literatura

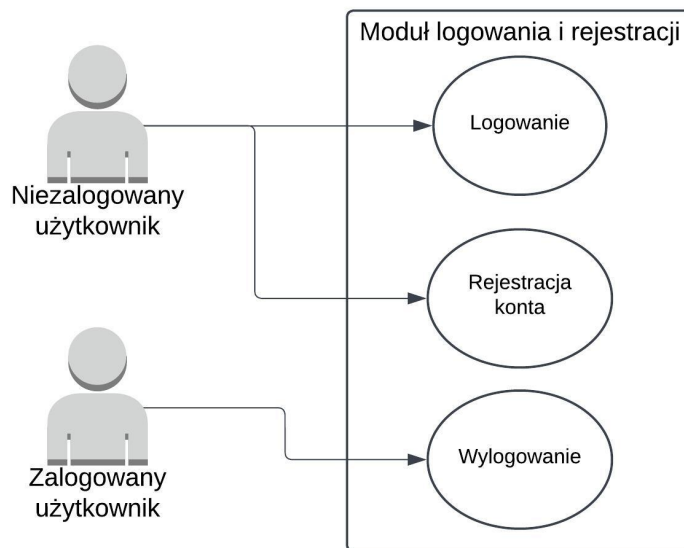
- [1] statista.com. [Online] <https://www.statista.com/topics/876/android/#topicOverview>, data dostępu 15.01.2025
- [2] gradle.org. [Online] <https://gradle.org/>, data dostępu 27.01.2025
- [3] developer.android.com. [Online] <https://developer.android.com/studio/intro?hl=pl>, data dostępu 15.01.2025
- [4] developer.android.com. [Online] <https://developer.android.com/kotlin/overview>, data dostępu 15.01.2025
- [5] developer.android.com. [Online] <https://developer.android.com/develop/ui/compose/tutorial>, data dostępu 21.01.2025
- [6] docs.mapbox.com. [Online] <https://docs.mapbox.com/help/getting-started/#introduction-to-the-mapbox-platform>, data dostępu 27.01.2025
- [7] laravel.com. [Online] <https://laravel.com/docs/11.x>, data dostępu 16.01.2025
- [8] dev.mysql.com. [Online] <https://dev.mysql.com/doc/>, data dostępu 16.01.2025
- [9] getcomposer.org. [Online] <https://getcomposer.org/doc/>, data dostępu 16.01.2025
- [10] docs.docker.com. [Online] <https://docs.docker.com/>, data dostępu 16.01.2025
- [11] swagger.io. [Online] <https://swagger.io/docs/>, data dostępu 16.01.2025
- [12] Booking.com. "O Booking.com". [Online] <https://www.booking.com/content/about.pl.html>, data dostępu 02.11.2024
- [13] Airbnb.pl. [Online] <https://www.airbnb.pl/>, data dostępu 02.11.2024
- [14] Instagram.com. [Online] <https://about.instagram.com/>, data dostępu 02.11.2024
- [15] Snapchat. [Online] <https://play.google.com/store/apps/details?id=com.snapchat.android>, data dostępu 02.11.2024
- [16] Tripadvisor.com. [Online] <https://tripadvisor.mediaroom.com/us-about-us>, data dostępu 02.11.2024
- [17] lucidchart.com. [Online] <https://www.lucidchart.com/pages/pl/diagram-przypadek%C3%B3w-u%C5%BCycia-UML>, data dostępu 02.11.2024
- [18] docs.mapbox.com. [Online] <https://docs.mapbox.com/android/maps/guides/install/>, data dostępu 27.01.2025

- [19] testerzy.pl. [Online] <https://testerzy.pl/baza-wiedzy/artykuly/testowanie-manualne-i-automatyczne-roznice>, data dostępu 21.01.2025
- [20] ralphjsmit.com. [Online] <https://ralphjsmit.com/laravel-mock-dependencies>, data dostępu 24.01.2025
- [21] developer.android.com. [Online] <https://developer.android.com/tools/logcat>, data dostępu 22.01.2025
- [22] clickup.com. [Online] <https://clickup.com/>, data dostępu 28.01.2025

Dodatek A

Diagramy przypadków użycia

Na rys. A.1 zaprezentowano diagram przypadków użycia związany z modulem logowania i rejestracji. Moduł umożliwia użytkownikowi dostęp do aplikacji oraz tworzenie nowego konta w systemie.



RYSUNEK A.1: Diagram przypadków użycia dla modułu logowania i rejestracji

A1. Logowanie

Aktor: Niezalogowany użytkownik

Scenariusz główny:

1. Użytkownik wypełnia formularz logowania, podaje adres e-mail, hasło.
2. System weryfikuje poprawność podanych danych.
3. System przenosi użytkownika na stronę główną aplikacji.

A2. Rejestracja konta

Aktor: Niezalogowany użytkownik

Scenariusz główny:

1. Użytkownik wypełnia formularz rejestracji, podaje nazwę użytkownika, adres e-mail oraz hasło.
2. System weryfikuje poprawność podanych danych.
3. System przenosi użytkownika na stronę główną aplikacji.

A3. Wylogowanie

Aktor: Zalogowany użytkownik

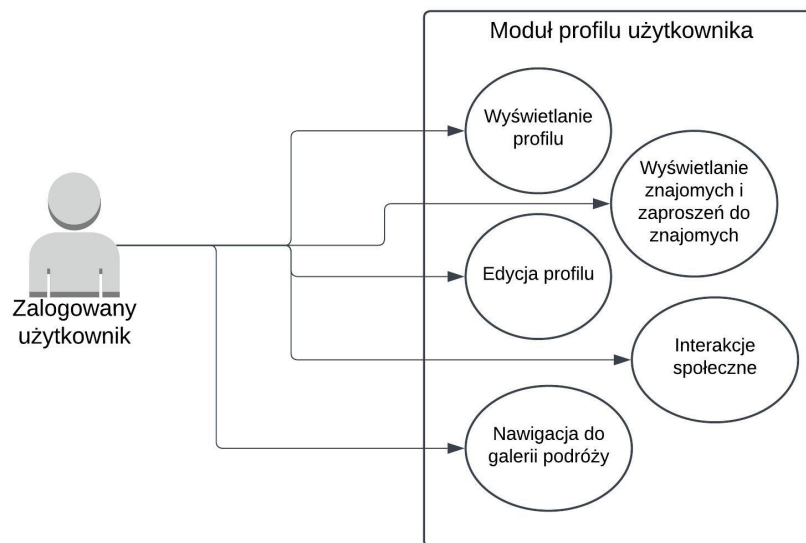
Scenariusz główny:

1. Użytkownik wybiera opcję "Logout" z górnego paska nawigacyjnego aplikacji.
2. System wyświetla okno dialogowe z prośbą o potwierdzenie wylogowania.
3. Użytkownik potwierdza wylogowanie z konta.
4. System wylogowuje użytkownika z konta i przenosi do ekranu logowania.

Scenariusz alternatywny:

1. Użytkownik wychodzi z okna dialogowego i pozostaje zalogowany w aplikacji.

Rys. A.2 przedstawia przypadki użycia, które występują w module dotyczącym profilu użytkownika. Moduł zawiera takie funkcjonalności jak edycja profilu użytkownika oraz wyświetlanie profilu użytkownika.



RYSUNEK A.2: Diagram przypadków użycia dla modułu profilu użytkownika

B1. Wyświetlanie profilu użytkownika

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik przechodzi do ekranu swojego profilu lub profilu innego użytkownika aplikacji.
2. Aplikacja pobiera z serwera informacje o profilu użytkownika.

3. Aplikacja wyświetla profil użytkownika.

Scenariusz alternatywny:

1. System informuje użytkownika o niepowodzeniu w procesie uwierzytelnienia w przypadku wysłania niepoprawnego tokenu.
2. Jeżeli wystąpił błąd podczas pobierania danych z serwera, aplikacja wyświetla odpowiedni komunikat.

B2. Edycja profilu użytkownika

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik przechodzi do ekranu edycji profilu.
2. Aplikacja pobiera z serwera informacje o profilu użytkownika.
3. Użytkownik edytuje poszczególne dane profilu.
4. System weryfikuje poprawność podanych danych.
5. System zapisuje dane w bazie danych.
6. Aplikacja przekazuje informacje o przeprowadzonej operacji edycji.

Scenariusz alternatywny:

1. Jeśli dane są niepoprawne, posiadają niepoprawny format np. liczba znaków w opisie jest za duża to aplikacja komunikuje użytkownika odpowiednim komunikatem zwrotnym.
2. System informuje użytkownika o niepowodzeniu w procesie uwierzytelnienia w przypadku wysłania błędnego tokenu.

B3. Nawigacja do galerii podróży

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik przeglądający profil przechodzi do galerii podróży, za pomocą odpowiedniego przycisku.
2. Aplikacja sprawdza czy użytkownik posiada dostęp do wyświetlenia galerii podróży.
3. Jeśli użytkownik posiada dostęp, to aplikacja pobiera z serwera informacje o zawartości galerii użytkownika.
4. Aplikacja wyświetla wybraną listę podróży.

Scenariusz alternatywny:

1. System informuje użytkownika o niepowodzeniu w procesie uwierzytelnienia w przypadku wysłania błędnego tokenu.
2. System informuje użytkownika o braku dostępu do galerii użytkownika, który nie należy do znajomych zalogowanego użytkownika aplikacji.

B4. Wyświetlanie znajomych i zaproszeń do znajomych

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik znajdujący się na swoim profilu klika w odpowiedni przycisk w celu wyświetlenia znajomych lub zaproszeń do znajomych.
2. Aplikacja wyświetla wybraną zawartość dotyczącą znajomych lub zaproszeń do znajomych.

B5. Interakcje społeczne

Aktor: Zalogowany użytkownik

Scenariusz główny: Akceptacja zaproszenia

1. Zalogowany użytkownik przechodzi do listy zaproszeń do znajomych znajdującej się w jego profilu.
2. Aplikacja wyświetla pobraną listę zaproszeń oraz dostępne opcje związane z interakcjami społecznymi - akceptację, usunięcie zaproszenia.
3. Zalogowany użytkownik akceptuje zaproszenie od wybranego użytkownika.
4. System dodaje zapraszającego użytkownika do listy znajomych zalogowanego użytkownika.

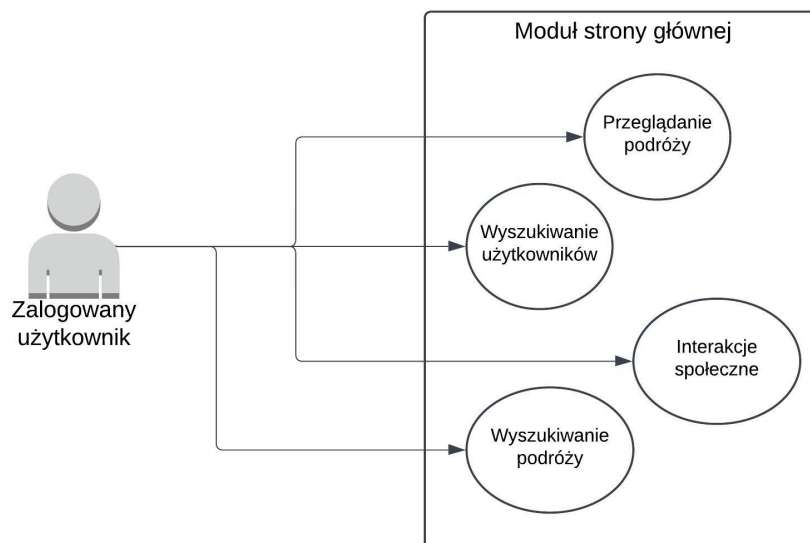
Scenariusz alternatywny: Usunięcie zaproszenia

1. Zalogowany użytkownik przechodzi do listy zaproszeń do znajomych znajdującej się w jego profilu.
2. Aplikacja wyświetla pobraną listę zaproszeń oraz dostępne opcje związane z interakcjami społecznymi - akceptację, usunięcie zaproszenia.
3. Zalogowany użytkownik usuwa zaproszenie od wybranego użytkownika.
4. System usuwa zaproszenie.

Scenariusz alternatywny: Usunięcie znajomego

1. Zalogowany użytkownik przechodzi do listy znajomych znajdującej się w jego profilu.
2. Aplikacja wyświetla pobraną listę znajomych.
3. Użytkownik wybiera opcje usunięcia znajomego.
4. System usuwa danego użytkownika z listy znajomych.

Rys. A.3 przedstawia przypadki użycia związane ze stroną główną aplikacji.



RYSUNEK A.3: Diagram przypadków użycia dla modułu strony głównej

C1. Przeglądanie podróży

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Aplikacja pobiera z serwera podróże użytkowników, którzy są znajomymi zalogowanego użytkownika.
2. Aplikacja wyświetla pobraną listę podróży.
3. Zalogowany użytkownik przechodzi do podglądu danej podróży.
4. Aplikacja pobiera z serwera więcej informacji o podróży m.in listę miejsc.
5. Użytkownik przechodzi do interaktywnej mapy, która wyświetla szczegóły podróży wraz z lokalizacją.

C2. Wyszukiwanie użytkowników

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik przechodzi do wyszukiwarki na stronie głównej.
2. Użytkownik wybiera odpowiedni typ wyszukiwania i wpisuje nazwę szukanego użytkownika.
3. Aplikacja wyświetla listę użytkowników o podanej nazwie wraz z dostępnymi opcjami - dodawanie do znajomych lub akceptacja, usuwanie zaproszenia.
4. Użytkownik może nawigować do interesującego go profilu.

Scenariusz alternatywny:

1. Użytkownik przechodzi do wyszukiwarki na stronie głównej.
2. Użytkownik wybiera odpowiedni typ wyszukiwania i wpisuje nazwę szukanego użytkownika.

3. Aplikacja wyświetla pustą listę w przypadku braku użytkowników o podanej nazwie.

C3. Wyszukiwanie podróży

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik przechodzi do wyszukiwarki na stronie głównej.
2. Użytkownik wybiera odpowiedni typ wyszukiwania i dostosowuje filtry wyszukiwania - data rozpoczęcia i zakończenia podróży, typ sortowania.
3. Aplikacja wyświetla listę podróży znajomych użytkownika.
4. Zalogowany użytkownik przechodzi do podglądu danej podróży.
5. Aplikacja pobiera z serwera więcej informacji o podróży m.in listę miejsc.

Scenariusz alternatywny:

1. Użytkownik przechodzi do wyszukiwarki na stronie głównej.
2. Użytkownik wybiera odpowiedni typ wyszukiwania i dostosowuje filtry wyszukiwania - data rozpoczęcia i zakończenia podróży, typ sortowania.
3. Aplikacja wyświetla pustą listę podróży w przypadku braku podróży o szukanych parametrach lub w przypadku braku znajomych.

C4. Interakcje społeczne

Aktor: Zalogowany użytkownik

Scenariusz główny: Akceptacja zaproszenia

1. Aplikacja wyświetla wynik wyszukiwania w postaci listy użytkowników oraz dostępne opcje związane z interakcjami społecznymi - dodanie do znajomych lub akceptację, usunięcie zaproszenia.
2. Zalogowany użytkownik akceptuje zaproszenie od wybranego użytkownika.
3. System dodaje zapraszającego użytkownika do listy znajomych zalogowanego użytkownika.

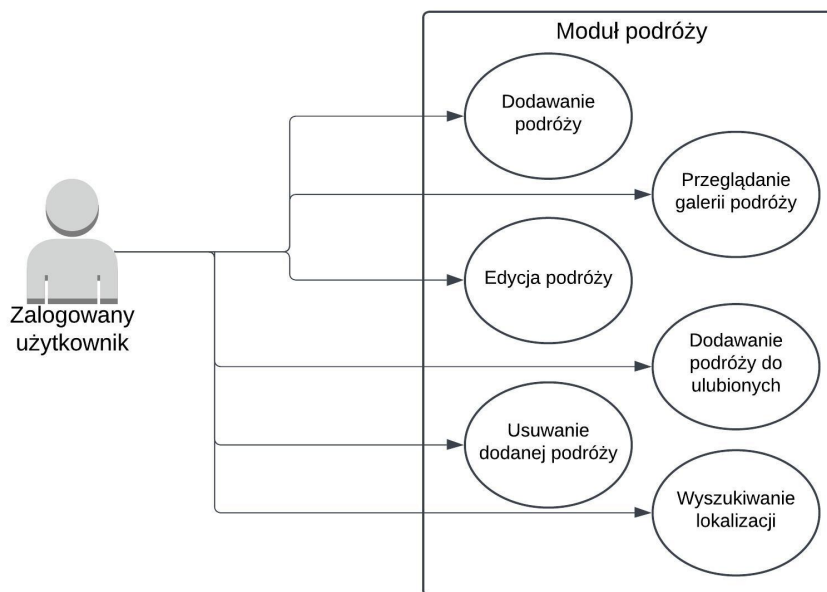
Scenariusz alternatywny: Usunięcie zaproszenia

1. Aplikacja wyświetla wynik wyszukiwania w postaci listy użytkowników oraz dostępne opcje związane z interakcjami społecznymi - dodanie do znajomych lub akceptację, usunięcie zaproszenia.
2. Zalogowany użytkownik usuwa zaproszenie od wybranego użytkownika.
3. System usuwa zaproszenie do znajomych.

Scenariusz alternatywny: Wysłanie zaproszenia

1. Aplikacja wyświetla wynik wyszukiwania w postaci listy użytkowników oraz dostępne opcje związane z interakcjami społecznymi - dodanie do znajomych lub akceptację, usunięcie zaproszenia.
2. Zalogowany użytkownik wybiera opcje dodania danego użytkownika do znajomych.
3. System przesyła od zalogowanego użytkownika zaproszenie do znajomych do konkretnego użytkownika aplikacji.

Rys. A.4 przedstawia diagram przypadków użycia dla modułu podróży. Moduł zawiera takie funkcjonalności jak dodawanie podróży, edycja podróży, usuwanie podróży, przeglądanie galerii podróży użytkownika.



RYСУNEK A.4: Diagram przypadków użycia dla modułu podróży

D1. Dodawanie podróży

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik przechodzi do formularza dodawania podróży.
2. Użytkownik wprowadza dane wymagane do dodania nowej podróży.
3. Użytkownik dodaje podróż, aplikacja wysła dane do serwera.
4. System weryfikuje wprowadzone dane.
5. Podróż zostaje zapisana w bazie danych, wpis podróży jest dostępny w galerii użytkownika.

Scenariusz alternatywny:

1. System informuje użytkownika o niepowodzeniu w procesie uwierzytelnienia w przypadku wysłania błędnego tokenu.
2. Jeśli dane są niepoprawne to aplikacja informuje użytkownika odpowiednią informacją zwrotną.

D2. Edycja podróży

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik wybiera istniejącą podróż z swojej galerii i przechodzi do formularza edycji podróży.

2. Użytkownik modyfikuje dowolne pola związane z podróżą lub miejscem.
3. Użytkownik zapisuje zmiany, aplikacja wysyła dane do serwera.
4. System weryfikuje poprawność danych.
5. Podróż zostaje zaktualizowana.

Scenariusz alternatywny:

1. System informuje użytkownika o niepowodzeniu w procesie uwierzytelnienia w przypadku wysłania błędnego tokenu.
2. Jeśli dane są niepoprawne to system komunikuje użytkownika odpowiednim komunikatem zwrotnym.

D3. Usuwanie podróży

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik wybiera istniejącą podróż z swojej galerii i wybiera opcję usuwania podróży.
2. Aplikacja wyświetla okno dialogowe z prośbą o potwierdzenie usunięcia podróży.
3. Użytkownik potwierdza usunięcie.
4. System usuwa podróż z bazy danych.

Scenariusz alternatywny:

1. Aplikacja wyświetla okno dialogowe z prośbą o potwierdzenie usunięcia podróży.
2. Użytkownik zamyka okno dialogowe.
3. Podróż pozostaje w bazie danych.

D4. Dodawanie podróży do ulubionych

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik wybiera istniejącą podróż z swojej galerii i klika w przycisk odpowiedzialny za dodanie do ulubionych.
2. System dodaje podróż do listy ulubionych.

Scenariusz alternatywny:

1. Użytkownik wybiera istniejącą podróż z swojej galerii, która należy do listy ulubionych i klika w przycisk odpowiedzialny za dodanie do ulubionych.
2. System usuwa podróż z listy ulubionych.

D5. Przeglądanie galerii podróży

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik przechodzi z profilu do swojej galerii podróży lub galerii znajomego.
2. Aplikacja pobiera zakończone, ulubione, zaplanowane podróże danego użytkownika.
3. Aplikacja wyświetla pobrane z serwera podróże w galerii.

4. Zalogowany użytkownik przechodzi do podglądu danej podróży.
5. Aplikacja pobiera z serwera więcej informacji o podróży m.in listę miejsc.
6. Użytkownik przechodzi do interaktywnej mapy, która wyświetla szczegóły podróży wraz z lokalizacją.

Scenariusz alternatywny:

1. Użytkownik przechodzi z profilu do galerii podróży użytkownika.
2. Aplikacja informuje zalogowanego użytkownika, że nie ma dostępu do galerii (w przypadku gdy dany użytkownik nie należy do znajomych zalogowanego użytkownika).

D6. Wyszukiwanie lokalizacji

Aktor: Zalogowany użytkownik

Scenariusz główny:

1. Użytkownik przechodzi z formularza podróży do ekranu wyszukiwania podróży Search-MapScreen.
2. Użytkownik wpisuje adres szukanego miejsca.
3. API Mapboxa zwraca odpowiedź z nazwą i współrzędnymi lokalizacji.
4. Mapa wczytuje znacznik i adres lokalizacji.
5. Użytkownik zapisuje wyszukaną lokalizację.

Scenariusz alternatywny: Ręczne ustawienie lokalizacji

1. Użytkownik przechodzi z formularza podróży do ekranu wyszukiwania podróży Search-MapScreen.
2. Użytkownik szuka wybranego miejsca na mapie i klika na mapę w wybrany punkt.
3. Mapa wczytuje znacznik i adres lokalizacji.
4. Użytkownik zapisuje ręcznie ustawioną lokalizację.



© 2025 Adrian Oblizajek, Rafał Grudziak

Instytut Radiokomunikacji, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.